
EBlink User Manual

Release 6.3

Jun 27, 2026

Gerard Zagma

CONTENTS

1	Introduction	1
1.1	Features	2
1.2	Supported probes	3
1.3	How it works	4
1.4	This manual	5
2	Usage	6
2.1	Before you start	7
2.2	Your first command	8
2.3	Saving yourself some typing	9
2.4	The command line structure	10
2.5	Configuring the probe: -I	11
2.6	Loading a script: -S	12
2.7	Flashing firmware: -F	13
2.8	Debugging with GDB: -G	14
2.8.1	Combining -F and -G	14
2.8.2	GDB monitor commands	15
2.9	Configuring the Cortex-M core: -T	17
2.10	Other options	18
3	The script language	19
3.1	Lexical Structure	20
3.1.1	Identifiers	20
3.1.2	Keywords	20
3.1.3	Operators	20
3.1.4	Other tokens	20
3.1.5	Literals	20
3.1.6	Comments	21
3.2	Values and Data types	22
3.2.1	Integer	22
3.2.2	Float	22
3.2.3	String	22
3.2.4	Null	23
3.2.5	Bool	23
3.2.6	Table	23
3.2.7	Array	23
3.2.8	Function	23
3.2.9	Class	23
3.2.10	Class Instance	24
3.2.11	Userdata	24
3.3	Execution Context	25
3.3.1	Variables	25
3.4	Statements	27
3.4.1	Block	27

3.4.2	Control Flow Statements	27
3.4.3	Loops	29
3.4.4	break	29
3.4.5	continue	29
3.4.6	return	30
3.4.7	Local variables declaration	30
3.4.8	Function declaration	30
3.4.9	Class declaration	30
3.4.10	try/catch	30
3.4.11	throw	31
3.4.12	const	31
3.4.13	enum	31
3.4.14	Expression statement	31
3.5	Expressions	32
3.5.1	Assignment	32
3.5.2	Operators	32
3.5.3	Table Constructor	35
3.5.4	clone	36
3.5.5	Array constructor	36
3.6	Tables	37
3.6.1	Construction	37
3.6.2	Slot creation	37
3.6.3	Slot deletion	37
3.7	Arrays	38
3.8	Functions	39
3.8.1	Function declaration	39
3.8.2	Function calls	41
3.8.3	Binding an environment to a function	41
3.8.4	Lambda Expressions	41
3.8.5	Free Variables	42
3.8.6	Tail Recursion	42
3.9	Classes	43
3.9.1	Class Declaration	43
3.9.2	Class Instances	46
3.9.3	Inheritance	47
3.9.4	Metamethods	49
3.10	Constants & Enumerations	50
3.10.1	Constants	50
3.10.2	Enumerations	50
3.10.3	Implementation notes	51
3.11	Built-in Functions	52
3.11.1	Global Symbols	52
3.11.2	Integer	52
3.11.3	Float	52
3.11.4	Bool	53
3.11.5	String	53
3.11.6	Table	54
3.11.7	Array	54
3.11.8	Function	56
3.11.9	Class	56
3.11.10	Class Instance	56
4	EBlink Scripting	57
4.1	Introduction	58
4.1.1	How scripts are loaded	58
4.1.2	CLI defines	58
4.1.3	API objects	58
4.1.4	Minimal example	58

4.2	Error Handling	60
4.2.1	Exception types	60
4.2.2	Return values	60
4.2.3	Propagating errors	61
4.2.4	Helper functions	61
4.2.5	User-defined error codes	61
4.3	Logging & Output	63
4.3.1	Output functions	63
4.3.2	Verbosity levels	63
4.3.3	Format strings	64
4.4	Global Functions	65
4.4.1	Script loading	65
4.4.2	Type checking	65
4.4.3	Data conversion	65
4.4.4	EBlink information	66
4.4.5	String utilities	66
4.4.6	System utilities	66
4.4.7	Math	67
4.5	Constants	68
4.5.1	VERSION	68
4.5.2	Target state	68
4.5.3	Reset types	68
4.5.4	Error codes	68
4.6	EBlink Interaction	70
4.6.1	Overview	70
4.6.2	Call order	70
4.6.3	Reference	71
4.7	EBbuffer	75
4.7.1	Construction	75
4.7.2	Memory access	75
4.7.3	Index operator	75
4.7.4	Buffer management	75
4.7.5	Trimming	76
4.7.6	Operators	76
4.7.7	Example	76
4.8	InterfAPI	77
4.8.1	Probe information	77
4.8.2	DAP access	77
4.8.3	Memory read	78
4.8.4	Memory write	78
4.8.5	File loading	78
4.9	DeviceAPI	79
4.9.1	Memory map	79
4.9.2	Memory map XML format	79
4.9.3	Flash management	80
4.10	TargetAPI	82
4.10.1	Target information	82
4.10.2	Execution control	82
4.10.3	Register access	83
4.10.4	Example: read and print all core registers	83
4.11	ServicesAPI	84
4.11.1	Service start behaviour	84
4.11.2	Example: dual-core GDB setup	84
4.11.3	Reference	85
4.12	Script Debugger	86
4.12.1	Enabling the debugger	86
4.12.2	Trace output format	86
4.12.3	Script breakpoints	86

4.12.4	Interactive commands	87
4.12.5	Call stack and locals	87
4.12.6	Tips	87
4.13	Addendum: Squirrel Extensions	88
4.13.1	Multiline verbatim strings	88
4.13.2	Compile-time constant expressions	88
4.14	API Quick Reference	90
	Index	93

INTRODUCTION

EBlink is an ARM Cortex-M debug and flash programming tool, available for Windows, Linux x86_64, and Raspberry Pi (32/64-bit). It connects to the target through a hardware debug probe and is fully driven by Squirrel scripts, making it easy to add support for new devices without recompiling the tool.

1.1 Features

Flash programming

Erases, programs, and verifies target flash from ELF, Intel HEX, or SREC files. Only modified sectors are re-programmed by default. In-place byte patching, memory compare, and memory dump are also supported.

GDB server

Exposes a GDB remote-serial-protocol server so debuggers (arm-none-eabi-gdb, IDEs) can connect for source-level debugging. Supports up to four concurrent GDB connections per instance, flash caching for fast re-flash, and full semihosting. Multi-core targets (e.g. STM32H7x5) are handled with a separate GDB server per core.

Hotplug / non-intrusive inspection

Connect to a running target without causing a reset. Read and write memory, registers, and flash while the target is executing.

Script-driven target support

Every device-specific action — memory map, flash erase, flash write, custom reset — is implemented in a Squirrel script. Device scripts for Silicon Labs, STMicro, Atmel, NXP, and Renesas targets are included. Auto-detection is available via `-S auto`.

Exception unwinding

Fault stack frame unwinding with visual notification, configurable per target.

Script debugging

Breakpoints and variable watches in device scripts (since v6.0).

Real-time monitor (EBmon)

Standalone host-side monitor for non-GDB target control and background data logging (included since v6.21).

Remote debugging

GDB proxy support via EBridge for debugging across NAT networks or over Wi-Fi from a Raspberry Pi.

1.2 Supported probes

- **STlink V2 / V3** — including clones

1.3 How it works

At startup EBLink:

1. Opens the debug probe specified with `-I`.
2. Loads the target script(s) specified with `-S`.
3. Calls `ebMain()` in the script, which sets up the memory map and initialises the device.
4. Carries out the requested operation — flash programming, GDB server, script execution, or a combination.

1.4 This manual

Usage

All command-line options, interface options, flash options, and GDB server options.

The script language

The Squirrel scripting language: syntax, types, control flow, and built-in functions.

EBlink Scripting

The EBLink scripting API: interface, device, target and buffer objects, error handling, constants, and the callback reference.

CHAPTER TWO

USAGE

This chapter walks you through using ELink from scratch — from connecting your first probe to running a GDB debug session.

2.1 Before you start

You need two things before EBlind can do anything:

1. A **debug probe** connected to your PC and to the target board (SWD or JTAG header).
2. A **device script** that tells EBlind about your target — its memory map, how to erase and write flash, and how to reset it.

EBlind is shipped with device scripts for STMicro, NXP, Atmel, Silicon Labs and Renesas targets. If you are not sure which script to use, start with auto and EBlind will identify the connected device and select the right script for you.

2.2 Your first command

The minimal Eblink command is:

```
eblink -I stlink -S auto
```

-I stlink tells Eblink to use an STlink probe. -S auto tells Eblink to auto-detect your device and load the right script.

If Eblink starts without errors, your probe is connected and the target is recognised. Eblink will then exit immediately because you have not asked it to do anything yet — that is normal.

2.3 Saving yourself some typing

Typing `-I stlink -S auto` on every command gets old fast. EBLink reads two environment variables at startup so you can set your defaults once and leave them out of every command:

EB_DEFAULT_PROBE

Used as the `-I` argument when `-I` is not on the command line.

EB_DEFAULT_SCRIPT

Used as a `-S` argument when `-S` is not on the command line.

Set them in your shell profile or system settings:

```
export EB_DEFAULT_PROBE=stlink
export EB_DEFAULT_SCRIPT=auto
```

After this, `eblink -F file=firmware.elf` is enough to flash a board — EBLink picks up the probe and script from the environment. Anything you do specify on the command line overrides the environment variable, so you can always deviate from the default for a single session without changing the setting:

```
# override for this session with explicit probe options:
eblink -I stlink,speed=1000 -F file=firmware.elf
```

2.4 The command line structure

```
eblink -I <probe>[,options] -S <script> [operation] [options...]
```

EBlink is built from independent modules. Each command-line switch is routed directly to its own module, so adding a new probe type or target architecture does not affect the rest of the tool:

```
-I ... --> [ Interface module(s) ]
-S ... --> [ Script engine       ]
-T ... --> [ Target module(s)    ]
-F ... --> [ Flash module        ]
-G ... --> [ GDB server          ]
```

For switches that can have multiple implementations — currently `-I` for probe types and `-T` for target architectures — the first parameter is always the module name, followed by optional comma-separated options for that module:

```
-I stlink[,...]
-T cortex-m[,...]
```

This makes it straightforward to add support for new probe types or target architectures in the future without changing the command-line structure.

EBlink always follows this sequence when it starts:

1. Open the debug probe (`-I`).
2. Load the device script(s) (`-S`).
3. Call `ebMain()` in the script — this sets up the memory map and initialises the target.
4. Carry out whatever operation you asked for (flash, GDB server, or both).

If any step fails, EBlink stops and reports the error. The most common first-time problems are a wrong probe type in `-I`, a missing USB driver, or a target that is not powered.

2.5 Configuring the probe: -I

The probe type comes first in the -I string, followed by comma-separated options:

```
-I stlink,speed=4000
```

The most useful options are:

speed=<kHz>

The SWD clock speed. The default is the maximum the probe supports, which is fine in most cases. If you get connection errors, try a lower speed such as `speed=1000` — long cables or noisy boards can cause problems at high speeds.

dr (disable reset)

Normally EBLink resets the target when it connects. Add `dr` to connect without a reset — useful when the target is already running code that must not be interrupted, or when the reset pin is not wired to the header.

swd / jtag

The debug protocol. SWD is the default and works on virtually all Cortex-M targets. Switch to `jtag` only if your hardware requires it.

serial=<sn>

If you have more than one probe plugged in at the same time, use the serial number to pick the right one. You can find serial numbers by running EBLink without this option — it will list all connected probes if there is ambiguity.

vcc=<V>

The target supply voltage (e.g. 3.3 or 1.8). EBLink uses this for speed calibration; it does not control the actual supply rail.

ap=<n>

By default EBLink scans the DAP and selects the first useful access port it finds. On multi-core devices such as the STM32H7, each core has its own AP, so you need to specify which one to use: `ap=0` for the CM7 core, `ap=1` for the CM4 core.

2.6 Loading a script: -S

The device script is what makes EBlind work with your specific target. It contains the memory map, the flash algorithm, and the reset strategy.

```
-S auto                # auto-detect the device
-S stm32.script        # explicit script by family
-S /home/user/scripts/my.script # explicit script by full path
```

With `auto`, EBlind reads the device ID register from the target and selects the matching script from the installed scripts. If the device is not recognised, EBlind tells you so and exits — in that case you need a custom script.

Multiple `-S` options are allowed. This makes it possible to combine a device family script with a project-specific script that adds board-level initialisation, custom flash regions, or helper functions:

```
eblink -I stlink -S stm32.script -S my_project.script
```

Functions defined in any loaded script can be called directly from the command line with `-E`, or interactively from a GDB session using GDB monitor commands (see *GDB monitor commands*).

Use `-P <path>` to add a directory to the script search path so you can refer to scripts by filename rather than full path:

```
eblink -I stlink -S my_target.script -P /home/user/my_scripts
```

Use `-D name=value` to pass a value into the script as a global variable. Some scripts use this for configuration that differs between board variants:

```
-D FLASH_SIZE=0x2000000
```

2.7 Flashing firmware: -F

The -F option programs, verifies, or inspects flash memory. You combine keywords and values with commas:

```
-F file=firmware.elf,erase,verify,run
```

The most important thing to understand is that by default — without `erase` — EBLink only re-programs flash sectors that differ from the new image. Unchanged sectors are left alone. This is faster and reduces flash wear. It works for most development workflows.

Use `erase` when you need a clean slate: for example when changing partition layouts, clearing old data sectors, or recovering a board with write-protected flash.

The most common operations:

file=<file>

The image to program. EBLink accepts ELF, Intel HEX, and SREC and detects the format automatically from the file contents.

erase

Chip-erase the entire flash before programming. Use this when the default smart re-flash is not sufficient.

verify

Read back every programmed sector and compare it with the image. If anything does not match, EBLink reports an error. Adds some time but gives you certainty.

run

Release the target and let it start running after all operations are done. Without this the target stays halted.

cmp=<file>

Compare flash contents with a file without changing anything. Useful for checking whether a board already has the right firmware.

write=<hex>@<address>

Patch bytes at a specific flash address. The value is written as raw hex — two hex digits per byte, no `0x` prefix. The address uses the normal `0x` notation. This is done *after* the file upload, so you can patch a serial number or device-specific calibration into an otherwise identical firmware image:

```
-F file=fw.elf,write=0102030405@0x0800FF00
```

Multiple write options are allowed in a single -F.

read=<length>@<address>

Read bytes from the target and print them as hex. Handy for checking a register value or a specific memory location without a full debug session. Use `-v 0` to suppress all other EBLink output so only the hex bytes appear:

```
eblink -I stlink -S auto -v 0 -F read=4@0x08000004
```

dump=<length>@<address>:<file>

Save a region of memory to a file. Use a `.hex` extension for Intel HEX format, any other extension for raw binary.

Typical flashing examples:

```
# Development: only re-program what changed
eblink -I stlink -S auto -F file=firmware.elf

# Production: erase everything, program, verify, start
eblink -I stlink -S auto -F erase,verify,run,file=firmware.elf

# Check if the board already has the right firmware
eblink -I stlink -S auto -F cmp=firmware.elf
```

2.8 Debugging with GDB: -G

Adding -G starts a GDB remote-serial-protocol server. You then point your debugger (arm-none-eabi-gdb, an IDE like EmBitz or VS Code) at that server and connect:

```
eblink -I stlink -S auto -G
```

By default EBLink listens on all network interfaces on TCP port 2331. Your GDB client connects with:

```
target remote localhost:2331
```

The most useful -G options:

port=<n>

Use a different TCP port. Useful when the default port 2331 is already in use, or when multiple GDB servers are started at the same time:

```
-G port=4242
```

ap=<n>

Attach this GDB server to a specific DAP access port. By default EBLink selects the first useful AP it finds. On a dual-core device (e.g. STM32H7) multiple -G options can be given — one per core:

```
-G ap=0,name=CM7,port=2331 -G ap=1,name=CM4,port=2332
```

Each GDB session then controls its own core independently.

Note: Device scripts for dual-core targets (such as the STM32H7 script) set up multiple GDB servers automatically via `ebServicesSetup()`. Multiple -G on the command line is only needed when the script does not handle this itself. See [ServicesAPI](#).

name=<name>

Optional instance name shown in the EBLink log. When a single -G is given the name defaults to GDB; with multiple -G it defaults to GDB_<ap> (e.g. GDB_0, GDB_1).

s

Shut down EBLink when GDB disconnects. Without this, EBLink keeps running and waits for the next connection. Add s in automated test flows where EBLink should exit cleanly after each session.

nc

Disable the flash cache. EBLink normally caches flash contents so that re-flashing from GDB is fast. If your target modifies its own flash at runtime (bootloaders, EEPROM emulation), disable the cache so EBLink always reads back the real flash contents.

2.8.1 Combining -F and -G

You do not need to flash anything to use the GDB server — -G works on its own and connects to whatever is already running on the target. Adding -F is useful when you want to write something to flash first, for example a bootloader, a certificate, or device-specific calibration data, and then hand control to the debugger:

```
eblink -I stlink -S auto -F file=bootloader.elf -G port=4242
```

EBlink programs the flash first, then starts the GDB server and waits for a connection.

2.8.2 GDB monitor commands

From any GDB session you can send commands directly to EBLink using the GDB monitor prefix:

```
(gdb) monitor <command>
```

The following built-in commands are available:

reset [**0|1|2**]

Reset the target. The optional argument selects the reset type: 0 = system reset (default), 1 = core-only reset, 2 = JTAG reset.

clrbp

Clear all software (flash) breakpoints.

is_running

Returns OK if the target is running, error if it is halted. IDEs such as EmBitz use this command to implement hotplug without requiring the extended GDB protocol — by querying the target state before attaching.

reg <name>[=<value>]

Read or write a CPU register by name (e.g. PC, R0, SP). Without a value the current register contents are returned. The value can be a literal (0x20001000) or an indirect memory read (R0=(0x20001000)).

writeu32 <addr>=<value>

Write a 32-bit value to a memory address. Both address and value accept decimal or hex notation:

```
monitor writeu32 0x40021000=0x00000001
```

semihosting [enable|disable]

Enable or disable ARM semihosting. Without an argument semihosting is enabled. Also accepted as EnableSemihost.

sleep <ms>

Pause EBLink for the given number of milliseconds before processing the next GDB request.

execute <function call>

Call a function defined in any loaded script, exactly as with -E on the command line:

```
monitor execute myBoardInit()
```

servicePollInterval <ms>

Adjust the GDB poll interval in milliseconds. Lower values give more responsive single-stepping at the cost of higher CPU load on the host.

DAP register access

The following commands provide direct access to the ARM Debug Access Port registers. They are compatible with the STMicro GDB server protocol and are used by IDEs such as STM32CubeIDE to identify and configure the target.

ReadDP <addr>

Read a Debug Port register. Address in hex:

```
monitor ReadDP 0x0
```

Register 0x0 returns the DP IDCODE (e.g. 0x2ba01477 for a Cortex-M with SWD).

WriteDP <addr> <value>

Write a value to a Debug Port register:

```
monitor WriteDP 0x8 0x00FF0000
```

ReadAP <ap> <addr>

Read an Access Port register. Register `0xF8` on AP 0 returns the MEM-AP IDR, which IDEs use to verify the target is an ARM device:

```
monitor ReadAP 0 0xf8
```

WriteAP <ap> <addr> <value>

Write a value to an Access Port register:

```
monitor WriteAP 0 0x0 0x22000040
```

WriteMemAP <ap> <addr> <value>

Write a 32-bit value to memory via a specific Access Port. Equivalent to `WriteAP` in this implementation.

Scripts can also intercept any monitor command by defining the function `ebGDBqRcmd(command, length, buffer)` — if it exists, EBlink calls it first and only falls through to the built-in handler if the function returns without writing to the buffer.

2.9 Configuring the Cortex-M core: -T

-T is optional and rarely needed for everyday use. EBlink auto-detects the core type. You add -T only when you want to change the default fault handling or reset behaviour.

fu=<0..2>

Fault unwind level. When your target crashes (HardFault, MemManage, etc.), EBlink can decode the exception stack frame and show you exactly which instruction faulted and why. 0 = disabled, 1 = passive (decode when halted), 2 = active (catch the fault immediately, default). This is one of EBlink's most useful debugging features — leave it at the default 2 unless you have a specific reason not to.

reset=<0..2>

Reset strategy: 0 = system reset (default), 1 = core-only reset, 2 = JTAG reset. Change this only if a system reset causes problems — for example when it reinitialises peripherals that need to stay powered.

fc=<flags>

Selects which exception types trigger the vector catch — the mechanism that halts the core the instant a fault fires, before any fault handler runs. Only active when fu=2 (the default).

Each letter enables one exception class:

Flag	Name	Catches when...
r	RESET	the core executes a reset (e.g. a watchdog or software reset)
m	MEM	a MemManage fault fires (MPU violation, null-pointer access)
n	NOCP	a coprocessor instruction is executed but no coprocessor is present
c	CHK	a UsageFault check error occurs (unaligned access, divide by zero)
s	STAT	a UsageFault invalid-state or invalid-PC error occurs
b	BUS	a BusFault fires (bad memory access on the bus)
i	INT	an error occurs during exception entry or return
h	HARD	a HardFault fires

The default is all flags (rmncsbih). The most common reason to change this is to omit r so that a target that deliberately resets itself (watchdog, bootloader) does not stop in the debugger every time:

```
-T cortex-m,fc=rmncsbih
```

2.10 Other options

-v <0..8>

Controls how much EBlind prints. The default is 4.

Level	Output
0	Errors only. Use for scripted calls where only <code>-F</code> read output matters.
1	Adds the version banner.
2	Adds operational messages (probe connected, flash progress) and animation.
3	Adds script <code>print()</code> output and yes/no prompts.
4	Adds target info: CPU type, flash regions, breakpoint count. (default)
5	Adds notices.
6	Adds warnings.
7	Adds internal debug messages. Animation is disabled at this level.
8	Adds low-level protocol trace for investigating probe communication.

For script debugging, use `-v 7` for function-level tracing or `-v 8` for line-level tracing.

-H

Connect without resetting the target — same as adding `dr` to the `-I` string.

-E <call>

Execute a Squirrel function after initialisation. Lets you call script functions from the command line without writing a wrapper script:

```
-E "dataflash_erase(0x08000500)"
```

Multiple `-E` options are executed in order.

-Q <n|y|d>

How EBlind handles yes/no questions from a script: `n` = always no, `y` = always yes, `d` = use the script default. Useful for unattended automation where no one is at the keyboard.

THE SCRIPT LANGUAGE

EBlink uses **Squirrel** as its scripting language — a high-level, imperative, object-oriented language designed by Alberto Demichelis as a lightweight yet powerful scripting tool. The language home page and full reference are at squirrel-lang.org.

Squirrel offers dynamic typing, higher-order functions, tail recursion, exception handling, and automatic memory management, with both compiler and virtual machine fitting in about 6k lines of C++ code.

3.1 Lexical Structure

3.1.1 Identifiers

Identifiers start with an alphabetic character or the symbol ‘_’ followed by any number of alphabetic characters, ‘_’ or digits ([0-9]). Squirrel is a case sensitive language meaning that the lowercase and uppercase representation of the same alphabetic character are considered different characters. For instance, “foo”, “Foo” and “fOo” are treated as 3 distinct identifiers.

3.1.2 Keywords

The following words are reserved and cannot be used as identifiers:

base	break	case	catch	class	clone
continue	const	default	delete	else	enum
extends	for	foreach	function	if	in
local	null	resume	return	switch	this
throw	try	typeof	while	yield	constructor
instanceof	true	false	static	__LINE__	__FILE__

Keywords are covered in detail later in this document.

3.1.3 Operators

Squirrel recognizes the following operators:

!	!=		==	&&	>=	<=	>
<=>	+	+=	-	-=	/	/=	*
*=	%	%=	++	--	<-	=	&
^		~	>>	<<	>>>		

3.1.4 Other tokens

Other significant tokens are:

{	}	[	]	.	:
::	'	;	"	@	"

3.1.5 Literals

Squirrel accepts integer numbers, floating point numbers and string literals.

34	Integer number(base 10)
0xFF00A120	Integer number(base 16)
0753	Integer number(base 8)
'a'	Integer number
1.52	Floating point number
1.e2	Floating point number
1.e-2	Floating point number
"I'm a string"	String
@"I'm a verbatim string"	String
@" I'm a multiline verbatim string "	String

3.1.6 Comments

A comment is text that the compiler ignores but that is useful for programmers. Comments are normally used to embed annotations in the code. The compiler treats them as white space.

A comment can be `/*` (slash, asterisk) characters, followed by any sequence of characters (including new lines), followed by the `*/` characters. This syntax is the same as ANSI C.:

```
/*
this is
a multiline comment.
this lines will be ignored by the compiler
*/
```

A comment can also be `//` (two slashes) characters, followed by any sequence of characters. A new line not immediately preceded by a backslash terminates this form of comment. It is commonly called a “*single-line comment*.”:

```
//this is a single line comment. this line will be ignored by the compiler
```

The character `#` is an alternative syntax for single line comment.:

```
# this is also a single line comment.
```

This to facilitate the use of squirrel in UNIX-style shell scripts.

3.2 Values and Data types

While Squirrel is a dynamically typed language and variables do not have a type, different operations may interpret the variable as containing a type. Squirrel's basic types are integer, float, string, null, table, array, function, class, instance, bool and userdata.

3.2.1 Integer

An Integer represents a 32 bit (or better) signed number.:

```
local a = 123 //decimal
local b = 0x0012 //hexadecimal
local c = 075 //octal
local d = 'w' //char code
```

3.2.2 Float

A float represents a 32 bit (or better) floating point number.:

```
local a=1.0
local b=0.234
```

3.2.3 String

Strings are an immutable sequence of characters. In order to modify a string is it necessary create a new one.

Squirrel's strings are similar to strings in C or C++. They are delimited by quotation marks(") and can contain escape sequences (\t, \a, \b, \n, \r, \v, \f, \\, \", \', \0, \x<hh>, \u<hhhh> and \U<hhhhhhhh>).

Verbatim string literals do not interpret escape sequences. They begin with @" and end with the matching quote. Verbatim string literals also can extend over a line break. If they do, they include any white space characters between the quotes:

```
local a = "I'm a wonderful string\n"
// has a newline at the end of the string
local x = @"I'm a verbatim string\n"
// the \n is literal, similar to "\\n" in a regular string.
```

However, a doubled quotation mark within a verbatim string is replaced by a single quotation mark:

```
local multiline = @"
    this is a multiline string
    it will ""embed"" all the new line
    characters
"
```

3.2.4 Null

The null value is a primitive value that represents the null, empty, or non-existent reference. The type Null has exactly one value, called null.:

```
local a = null
```

3.2.5 Bool

Bool is a double-valued (Boolean) data type. Its literals are `true` and `false`. A bool value expresses the validity of a condition (tells whether the condition is true or false).:

```
local a = true;
```

3.2.6 Table

Tables are associative containers implemented as a set of key/value pairs called slots.:

```
local t={}
local test=
{
  a=10
  b=function(a) { return a+1; }
}
```

3.2.7 Array

Arrays are simple sequence of objects. Their size is dynamic and their index always starts from 0.:

```
local a = ["I'm","an","array"]
local b = [null]
b[0] = a[2];
```

3.2.8 Function

Functions are similar to those in other C-like languages with a few key differences (see below).

3.2.9 Class

Classes are associative containers implemented as sets of key/value pairs. Classes are created through a 'class expression' or a 'class statement'. class members can be inherited from another class object at creation time. After creation, members can be added until an instance of the class is created.

3.2.10 Class Instance

Class instances are created by calling a *class object*. Instances, as tables, are implemented as sets of key/value pairs. Instance members cannot be dynamically added or removed; however the value of the members can be changed.

3.2.11 Userdata

Userdata objects are blobs of memory or pointers defined by the host application but stored within Squirrel variables.

3.3 Execution Context

The execution context is the union of the function stack frame and the function environment object(`this`) and the function root(`root table`). The stack frame is the portion of stack where the local variables declared in its body are stored. The environment object is an implicit parameter that is automatically passed by the function caller (see [functions](#)). The root table is a table associated to the function during its creation. The root table value of a function is the root table of the VM at the function creation. The root table of function can also be changed after creation with `closure.setroot()`. During the execution, the body of a function can only transparently refer to his execution context. This mean that a single identifier can refer to a local variable, to an environment object slot or to the slot of the closure root table; The environment object can be explicitly accessed by the keyword `this`. The closure root table can be explicitly accessed through the operator `::` (see [Variables](#)).

3.3.1 Variables

There are two types of variables in Squirrel, local variables and tables/arrays slots. Because global variables(variables stored in the root of a closure) are stored in a table, they are table slots.

A single identifier refers to a local variable or a slot in the environment object.:

```
derefexp := id;
```

```
_table["foo"]
_array[10]
```

with tables we can also use the `'.'` syntax:

```
derefexp := exp '.' id
```

```
_table.foo
```

Squirrel first checks if an identifier is a local variable (function arguments are local variables) if not looks up the environment object (`this`) and finally looks up to the closure root.

For instance::

```
function testy(arg)
{
    local a=10;
    print(a);
    return arg;
}
```

in this case `'foo'` will be equivalent to `'this.foo'` or `this["foo"]`.

Global variables are stored in a table called the root table. Usually in the global scope the environment object is the root table, but to explicitly access the closure root of the function from another scope, the slot name must be prefixed with `'::'` (`::foo`).

For instance::

```
function testy(arg)
{
    local a=10;
    return arg+::foo;
}
```

accesses the variable `'foo'` in the closure root table.

Since Squirrel 3.1 each function has a weak reference to a specific root table, this can differ from the current VM root table.:

```
function test() {  
    foo = 10;  
}
```

is equivalent to write:

```
function test() {  
    if("foo" in this) {  
        this.foo = 10;  
    }else {  
        ::foo = 10;  
    }  
}
```

3.4 Statements

A squirrel program is a simple sequence of statements.:

```
stats := stat [';' | '\n'] stats
```

Statements in squirrel are comparable to the C-Family languages (C/C++, Java, C# etc...): assignment, function calls, program flow control structures etc.. plus some custom statement like yield, table and array constructors (All those will be covered in detail later in this document). Statements can be separated with a new line or ';' (or with the keywords case or default if inside a switch/case statement), both symbols are not required if the statement is followed by '}'.

3.4.1 Block

```
stat := '{' stats '}'
```

A sequence of statements delimited by curly brackets ({ }) is called block; a block is a statement itself.

3.4.2 Control Flow Statements

squirrel implements the most common control flow statements: if, while, do-while, switch-case, for, foreach

true and false

Squirrel has a boolean type (bool) however like C++ it considers null, 0(integer) and 0.0(float) as *false*, any other value is considered *true*.

if/else statement

```
stat:= 'if' '(' exp ')' stat ['else' stat]
```

Conditionally execute a statement depending on the result of an expression.:

```
if(a>b)
    a=b;
else
    b=a;
////
if(a==10)
{
    b=a+b;
    return a;
}
```

while statement

```
stat := 'while' '(' exp ')' stat
```

Executes a statement while the condition is true.:

```
function testy(n)
{
    local a=0;
    while(a<n) a+=1;

    while(1)
    {
        if(a<0) break;
        a-=1;
    }
}
```

do/while statement

```
stat := 'do' stat 'while' '(' expression ')'
```

Executes a statement once, and then repeats execution of the statement until a condition expression evaluates to false.:

```
local a=0;
do
{
    print(a+"\n");
    a+=1;
} while(a>100)
```

switch statement

```
stat := 'switch' '(' exp ')' '{'
'case' case_exp ':'
    stats
['default' ':'
    stats]
'{'
```

Switch is a control statement allows multiple selections of code by passing control to one of the case statements within its body. The control is transferred to the case label whose case_exp matches with exp if none of the case match will jump to the default label (if present). A switch statement can contain any number if case instances, if 2 case have the same expression result the first one will be taken in account first. The default label is only allowed once and must be the last one. A break statement will jump outside the switch block.

3.4.3 Loops

for

```
stat := 'for' '(' [initexp] ';' [condexp] ';' [incexp] ')' statement
```

Executes a statement as long as a condition is different than false.:

```
for(local a=0;a<10;a+=1)
    print(a+"\n");
//or
glob <- null
for(glob=0;glob<10;glob+=1){
    print(glob+"\n");
}
//or
for(;;){
    print(loops forever+"\n");
}
```

foreach

```
'foreach' '(' [index_id',' value_id 'in' exp ']' stat
```

Executes a statement for every element contained in an array, table, class, string or generator. If exp is a generator it will be resumed every iteration as long as it is alive; the value will be the result of 'resume' and the index the sequence number of the iteration starting from 0.:

```
local a=[10,23,33,41,589,56]
foreach(idx,val in a)
    print("index="+idx+" value="+val+"\n");
//or
foreach(val in a)
    print("value="+val+"\n");
```

3.4.4 break

```
stat := 'break'
```

The break statement terminates the execution of a loop (for, foreach, while or do/while) or jumps out of switch statement;

3.4.5 continue

```
stat := 'continue'
```

The continue operator jumps to the next iteration of the loop skipping the execution of the following statements.

3.4.6 return

```
stat := return [exp]
```

The return statement terminates the execution of the current function/generator and optionally returns the result of an expression. If the expression is omitted the function will return null.

3.4.7 Local variables declaration

```
initz := id [= exp] [',' initz]  
stat := 'local' initz
```

Local variables can be declared at any point in the program; they exist between their declaration to the end of the block where they have been declared. *EXCEPTION*: a local declaration statement is allowed as first expression in a for loop.:

```
for(local a=0;a<10;a+=1)  
  print(a);
```

3.4.8 Function declaration

```
funcname := id ['::id]  
stat := 'function' id ['::id']+ '(' args ')' stat
```

creates a new function.

3.4.9 Class declaration

```
memberdecl := id '=' exp [';'] | '[' exp ']' '=' exp [';'] | functionstat |  
↳ 'constructor' functionexp  
stat := 'class' derefexp ['extends' derefexp] '{'  
  [memberdecl]  
  '}'
```

creates a new class.

3.4.10 try/catch

```
stat := 'try' stat 'catch' '(' id ' ') stat
```

The try statement encloses a block of code in which an exceptional condition can occur, such as a runtime error or a throw statement. The catch clause provides the exception-handling code. When a catch clause catches an exception, its id is bound to that exception.

3.4.11 throw

```
stat := 'throw' exp
```

Throws an exception. Any value can be thrown.

3.4.12 const

```
stat := 'const' id '=' 'Integer | Float | StringLiteral
```

Declares a constant (see *Constants & Enumerations*).

3.4.13 enum

```
enumerations := ( 'id' '=' Integer | Float | StringLiteral ) [',']  
stat := 'enum' id '{' enumerations '}'
```

Declares an enumeration (see *Constants & Enumerations*).

3.4.14 Expression statement

```
stat := exp
```

In Squirrel every expression is also allowed as statement, if so, the result of the expression is thrown away.

3.5 Expressions

3.5.1 Assignment

```
exp := derefexp '=' exp
exp:= derefexp '<-' exp
```

squirrel implements 2 kind of assignment: the normal assignment(=):

```
a = 10;
```

and the “new slot” assignment.:

```
a <- 10;
```

The new slot expression allows to add a new slot into a table(see [Tables](#)). If the slot already exists in the table it behaves like a normal assignment.

3.5.2 Operators

?: Operator

```
exp := exp_cond '?' exp1 ':' exp2
```

conditionally evaluate an expression depending on the result of an expression.

Arithmetic

```
exp:= 'exp' op 'exp'
```

Squirrel supports the standard arithmetic operators +, -, *, / and %. Other than that is also supports compact operators (+, -, *, /, %) and increment and decrement operators(++ and --):

```
a += 2;
//is the same as writing
a = a + 2;
x++
//is the same as writing
x = x + 1
```

All operators work normally with integers and floats; if one operand is an integer and one is a float the result of the expression will be float. The + operator has a special behavior with strings; if one of the operands is a string the operator + will try to convert the other operand to string as well and concatenate both together. For instances and tables, `_toString` is invoked.

Relational

```
exp := 'exp' op 'exp'
```

Relational operators in Squirrel are : `==`, `<`, `<=`, `>`, `>=`, `!=`

These operators return true if the expression is false and a value different than true if the expression is true. Internally the VM uses the integer 1 as true but this could change in the future.

3 ways compare

```
exp := 'exp' <=> 'exp'
```

the 3 ways compare operator `<=>` compares 2 values A and B and returns an integer less than 0 if `A < B`, 0 if `A == B` and an integer greater than 0 if `A > B`.

Logical

```
exp := exp op exp
exp := '!' exp
```

Logical operators in Squirrel are : `&&`, `||`, `!`

The operator `&&` (logical and) returns null if its first argument is null, otherwise returns its second argument. The operator `||` (logical or) returns its first argument if is different than null, otherwise returns the second argument.

The `!` operator will return null if the given value to negate was different than null, or a value different than null if the given value was null.

in operator

```
exp := keyexp 'in' tableexp
```

Tests the existence of a slot in a table. Returns true if *keyexp* is a valid key in *tableexp*

```
local t=
{
    foo="I'm foo",
    [123]="I'm not foo"
}

if("foo" in t) dostuff("yep");
if(123 in t) dostuff();
```

instanceof operator

```
exp := instanceexp 'instanceof' classexp
```

Tests if a class instance is an instance of a certain class. Returns true if *instanceexp* is an instance of *classexp*.

typeof operator

```
exp:= 'typeof' exp
```

returns the type name of a value as string.:

```
local a={},b="squirrel"  
print(typeof a); //will print "table"  
print(typeof b); //will print "string"
```

Comma operator

```
exp:= exp ',' exp
```

The comma operator evaluates two expression left to right, the result of the operator is the result of the expression on the right; the result of the left expression is discarded.:

```
local j=0,k=0;  
for(local i=0; i<10; i++ , j++)  
{  
    k = i + j;  
}  
local a,k;  
a = (k=1,k+2); //a becomes 3
```

Bitwise Operators

```
exp:= 'exp' op 'exp'  
exp := '~' exp
```

Squirrel supports the standard C-like bitwise operators `&`, `|`, `^`, `~`, `<<`, `>>` plus the unsigned right shift operator `>>>`. The unsigned right shift works exactly like the normal right shift operator(`>>`) except for treating the left operand as an unsigned integer, so is not affected by the sign. Those operators only work on integer values; passing of any other operand type to these operators will cause an exception.

Operators precedence

<code>-</code> , <code>~</code> , <code>!</code> , <code>typeof</code> , <code>++</code> , <code>--</code>	highest
<code>/</code> , <code>*</code> , <code>%</code>	...
<code>+</code> , <code>-</code>	
<code><<</code> , <code>>></code> , <code>>>></code>	
<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>instanceof</code>	
<code>==</code> , <code>!=</code> , <code><=></code>	
<code>&</code>	
<code>^</code>	
<code>&&</code> , <code>in</code>	
<code>+=</code> , <code>=</code> , <code>-=</code> , <code>/=</code> , <code>*=</code> , <code>%=</code>	...
<code>,</code> (comma operator)	lowest

3.5.3 Table Constructor

```
tslots := ( 'id' '=' exp | '[' exp ']' '=' exp ) [',']
exp := '{' [tslots] '}'
```

Creates a new table.:

```
local a = {} //create an empty table
```

A table constructor can also contain slots declaration; With the syntax:

```
local a = {
    slot1 = "I'm the slot value"
}
```

An alternative syntax can be:

```
'[' exp1 ']' = exp2 [',']
```

A new slot with exp1 as key and exp2 as value is created:

```
local a=
{
    [1]="I'm the value"
}
```

Both syntaxes can be mixed:

```
local table=
{
    a=10,
    b="string",
    [10]={},
    function bau(a,b)
    {
        return a+b;
    }
}
```

The comma between slots is optional.

Table with JSON syntax

Since Squirrel 3.0 is possible to declare a table using JSON syntax(see <http://www.wikipedia.org/wiki/JSON>). the following JSON snippet:

```
local x = {
    "id": 1,
    "name": "Foo",
    "price": 123,
    "tags": ["Bar", "Eek"]
}
```

is equivalent to the following squirrel code:

```
local x = {
    id = 1,
```

(continues on next page)

(continued from previous page)

```
name = "Foo",  
price = 123,  
tags = ["Bar","Eek"]  
}
```

3.5.4 clone

```
exp := 'clone' exp
```

Clone performs shallow copy of a table, array or class instance (copies all slots in the new object without recursion).

After the new object is ready the `_cloned` metamethod is called.

When a class instance is cloned the constructor is not invoked(initializations must rely on ``_cloned`` instead

3.5.5 Array constructor

```
exp := '[' [explist] '']
```

Creates a new array.:

```
a <- [] //creates an empty array
```

Arrays can be initialized with values during the construction:

```
a <- [1,"string!",[],{}] //creates an array with 4 elements
```

3.6 Tables

Tables are associative containers implemented as pairs of key/value (called slot); values can be any possible type and keys any type except 'null'. Tables are squirrel's skeleton, delegation and many other features are all implemented through this type; even the environment, where "global" variables are stored, is a table (known as root table).

3.6.1 Construction

Tables are created through the table constructor

3.6.2 Slot creation

Adding a new slot in a existing table is done through the "new slot" operator <-; this operator behaves like a normal assignment except that if the slot does not exists it will be created.:

```
local a = {}
```

The following line will cause an exception because the slot named 'newslot' does not exist in the table 'a':

```
a.newslot = 1234
```

this will succeed:

```
a.newslot <- 1234;
```

or:

```
a[1] <- "I'm the value of the new slot";
```

3.6.3 Slot deletion

```
exp:= delete derefexp
```

Deletion of a slot is done through the keyword delete; the result of this expression will be the value of the deleted slot.:

```
a <- {
  test1=1234
  deleteme="now"
}

delete a.test1
print(delete a.deleteme); //this will print the string "now"
```

3.7 Arrays

An array is a sequence of values indexed by a integer number from 0 to the size of the array minus 1. Arrays elements can be obtained through their index.:

```
local a=["I'm a string", 123]
print(typeof a[0]) //prints "string"
print(typeof a[1]) //prints "integer"
```

Resizing, insertion, deletion of arrays and arrays elements is done through a set of standard functions (see *built-in functions*).

3.8 Functions

Functions are first class values like integer or strings and can be stored in table slots, local variables, arrays and passed as function parameters. Functions can be implemented in Squirrel or in a native language with calling conventions compatible with ANSI C.

3.8.1 Function declaration

Functions are declared through the function expression:

```
local a = function(a, b, c) { return a + b - c; }
```

or with the syntactic sugar:

```
function ciao(a,b,c)
{
    return a+b-c;
}
```

that is equivalent to:

```
this.ciao <- function(a,b,c)
{
    return a+b-c;
}
```

a local function can be declared with this syntactic sugar:

```
local function tuna(a,b,c)
{
    return a+b-c;
}
```

that is equivalent to:

```
local tuna = function(a,b,c)
{
    return a+b-c;
}
```

is also possible to declare something like:

```
T <- {}
function T::ciao(a,b,c)
{
    return a+b-c;
}

//that is equivalent to write

T.ciao <- function(a,b,c)
{
    return a+b-c;
}

//or
```

(continues on next page)

(continued from previous page)

```
T <- {  
  function ciao(a,b,c)  
  {  
    return a+b-c;  
  }  
}
```

Default Paramaters

Squirrel's functions can have default parameters.

A function with default parameters is declared as follows:

```
function test(a,b,c = 10, d = 20)  
{  
  . . .  
}
```

when the function *test* is invoked and the parameter *c* or *d* are not specified, the VM automatically assigns the default value to the unspecified parameter. A default parameter can be any valid squirrel expression. The expression is evaluated at runtime.

Function with variable number of paramaters

Squirrel's functions can have variable number of parameters(varargs functions).

A vararg function is declared by adding three dots (...) at the end of its parameter list.

When the function is called all the extra parameters will be accessible through the *array* called *vargv*, that is passed as implicit parameter.

vargv is a regular squirrel array and can be used accordingly.:

```
function test(a,b,...)  
{  
  for(local i = 0; i < vargv.len(); i++)  
  {  
    ::print("varparam "+i+" = "+vargv[i]+"\\n");  
  }  
  foreach(i,val in vargv)  
  {  
    ::print("varparam "+i+" = "+val+"\\n");  
  }  
}  
  
test("goes in a","goes in b",0,1,2,3,4,5,6,7,8);
```

3.8.2 Function calls

```
exp := derefexp '(' explist ')'
```

The expression is evaluated in this order: derefexp after the explist (arguments) and at the end the call.

A function call in Squirrel passes the current environment object *this* as a hidden parameter. But when the function was immediately indexed from an object, *this* shall be the object which was indexed, instead.

If we call a function with the syntax:

```
mytable.foo(x,y)
```

the environment object passed to 'foo' as *this* will be 'mytable' (since 'foo' was immediately indexed from 'mytable')

Whereas with the syntax:

```
foo(x,y) // implicitly equivalent to this.foo(x,y)
```

the environment object will be the current *this* (that is, propagated from the caller's *this*).

It may help to remember the rules in the following way:

`foo(x,y) —> this.foo(x,y)` `table.foo(x,y) —> call foo with (table,x,y)`

It may also help to consider why it works this way: it's designed to assist with object-oriented style. When calling 'foo(x,y)' it's assumed you're calling another member of the object (or of the file) and so should operate on the same object. When calling 'mytable.foo(x,y)' it's written plainly that you're calling a member of a different object.

3.8.3 Binding an environment to a function

while by default a squirrel function call passes as environment object 'this', the object where the function was indexed from. However, is also possible to statically bind an environment to a closure using the built-in method `closure.bindenv(env_obj)`. The method `bindenv()` returns a new instance of a closure with the environment bound to it. When an environment object is bound to a function, every time the function is invoked, its 'this' parameter will always be the previously bound environment. This mechanism is useful to implement callbacks systems similar to C# delegates.

Note: The closure keeps a weak reference to the bound environment object, because of this if the object is deleted, the next call to the closure will result in a null environment object.

3.8.4 Lambda Expressions

```
exp := '@' '(' paramlist ')' exp
```

Lambda expressions are a syntactic sugar to quickly define a function that consists of a single expression. This feature comes handy when functional programming patterns are applied, like map/reduce or passing a compare method to `array.sort()`.

here a lambda expression:

```
local myexp = @(a,b) a + b
```

that is equivalent to:

```
local myexp = function(a,b) { return a + b; }
```

a more useful usage could be:

```
local arr = [2,3,5,8,3,5,1,2,6];
arr.sort(@(a,b) a <=> b);
arr.sort(@(a,b) -(a <=> b));
```

that could have been written as:

```
local arr = [2,3,5,8,3,5,1,2,6];
arr.sort(function(a,b) { return a <=> b; } );
arr.sort(function(a,b) { return -(a <=> b); } );
```

other than being limited to a single expression lambdas support all features of regular functions. in fact are implemented as a compile time feature.

3.8.5 Free Variables

A free variable is a variable external from the function scope as is not a local variable or parameter of the function. Free variables reference a local variable from a outer scope. In the following example the variables 'testy', 'x' and 'y' are bound to the function 'foo':

```
local x=10,y=20
local testy="I'm testy"

function foo(a,b)
{
    ::print(testy);
    return a+b+x+y;
}
```

A program can read or write a free variable.

3.8.6 Tail Recursion

Tail recursion is a method for partially transforming a recursion in a program into an iteration: it applies when the recursive calls in a function are the last executed statements in that function (just before the return). If this happens the squirrel interpreter collapses the caller stack frame before the recursive call; because of that very deep recursions are possible without risk of a stack overflow.:

```
function loopy(n)
{
    if(n>0){
        ::print("n="+n+"\n");
        return loopy(n-1);
    }
}

loopy(1000);
```

3.9 Classes

Squirrel implements a class mechanism similar to languages like Java/C++/etc... however because of its dynamic nature it differs in several aspects. Classes are first class objects like integer or strings and can be stored in table slots local variables, arrays and passed as function parameters.

3.9.1 Class Declaration

A class object is created through the keyword 'class'. The class object follows the same declaration syntax of a table(see [Tables](#)) with the only difference of using ';' as optional separator rather than ','.

For instance:

```
class Foo {
  //constructor
  constructor(a)
  {
    testy = ["stuff",1,2,3,a];
  }
  //member function
  function PrintTesty()
  {
    foreach(i,val in testy)
    {
      ::print("idx = "+i+" = "+val+" \n");
    }
  }
  //property
  testy = null;
}
```

the previous code example is a syntactic sugar for:

```
Foo <- class {
  //constructor
  constructor(a)
  {
    testy = ["stuff",1,2,3,a];
  }
  //member function
  function PrintTesty()
  {
    foreach(i,val in testy)
    {
      ::print("idx = "+i+" = "+val+" \n");
    }
  }
  //property
  testy = null;
}
```

in order to emulate namespaces, it is also possible to declare something like this:

```
//just 2 regular nested tables
FakeNamespace <- {
```

(continues on next page)

(continued from previous page)

```
    Utils = {}
}

class FakeNamespace.Utils.SuperClass {
  constructor()
  {
    ::print("FakeNamespace.Utils.SuperClass")
  }
  function DoSomething()
  {
    ::print("DoSomething()")
  }
}

function FakeNamespace::Utils::SuperClass::DoSomethingElse()
{
  ::print("FakeNamespace::Utils::SuperClass::DoSomethingElse()")
}

local testy = FakeNamespace.Utils.SuperClass();
testy.DoSomething();
testy.DoSomethingElse();
```

After its declaration, methods or properties can be added or modified by following the same rules that apply to a table(operator <-):

```
//adds a new property
Foo.stuff <- 10;

//modifies the default value of an existing property
Foo.testy <- "I'm a string";

//adds a new method
function Foo::DoSomething(a,b)
{
  return a+b;
}
```

After a class is instantiated is no longer possible to add new properties however is possible to add or replace methods.

Static variables

Squirrel's classes support static member variables. A static variable shares its value between all instances of the class. Statics are declared by prefixing the variable declaration with the keyword `static`; the declaration must be in the class body.

Note: Statics are read-only.

```
class Foo {
  constructor()
  {
    //..stuff
  }
}
```

(continues on next page)

(continued from previous page)

```

name = "normal variable";
//static variable
static classname = "The class name is foo";
};

```

Class Attributes

Classes allow to associate attributes to it's members. Attributes are a form of metadata that can be used to store application specific informations, like documentations strings, properties for IDEs, code generators etc... Class attributes are declared in the class body by preceding the member declaration and are delimited by the symbol `</` and `/>`. Here an example:

```

class Foo </ test = "I'm a class level attribute" />{
  </ test = "freakin attribute" /> //attributes of PrintTesty
  function PrintTesty()
  {
    foreach(i,val in testy)
    {
      ::print("idx = "+i+" = "+val+" \n");
    }
  }
  </ flippy = 10 , second = [1,2,3] /> //attributes of testy
  testy = null;
}

```

Attributes are, matter of fact, a table. Squirrel uses `</ />` syntax instead of curly brackets `{}` for the attribute declaration to increase readability.

This means that all rules that apply to tables apply to attributes.

Attributes can be retrieved through the built-in function `classobj.getattributes(membername)` (see [built-in functions](#)), and can be modified/added through the built-in function `classobj.setattributes(membername, val)`.

the following code iterates through the attributes of all Foo members.:

```

foreach(member, val in Foo)
{
  ::print(member+"\n");
  local attr;
  if((attr = Foo.getattributes(member)) != null) {
    foreach(i,v in attr)
    {
      ::print("\t"+i+" = "+(typeof v)+"\n");
    }
  }
  else {
    ::print("\t<no attributes>\n")
  }
}

```

3.9.2 Class Instances

The class objects inherits several of the table's feature with the difference that multiple instances of the same class can be created. A class instance is an object that share the same structure of the table that created it but holds its own values. Class *instantiation* uses function notation. A class instance is created by calling a class object. Can be useful to imagine a class like a function that returns a class instance.:

```
//creates a new instance of Foo
local inst = Foo();
```

When a class instance is created its member are initialized *with the same value* specified in the class declaration. The values are copied verbatim, *no cloning is performed* even if the value is a container or a class instances.

Note: FOR C# and Java programmers:

Squirrel doesn't clone member's default values nor executes the member declaration for each instance(as C# or java).

So consider this example:

```
class Foo {
    myarray = [1,2,3]
    mytable = {}
}

local a = Foo();
local b = Foo();
```

In the snippet above both instances will refer to the same array and same table.To achieve what a C# or Java programmer would expect, the following approach should be taken.

```
class Foo {
    myarray = null
    mytable = null
    constructor()
    {
        myarray = [1,2,3]
        mytable = {}
    }
}

local a = Foo();
local b = Foo();
```

When a class defines a method called 'constructor', the class instantiation operation will automatically invoke it for the newly created instance. The constructor method can have parameters, this will impact on the number of parameters that the *instantiation operation* will require. Constructors, as normal functions, can have variable number of parameters (using the parameter ...):

```
class Rect {
    constructor(w,h)
    {
        width = w;
        height = h;
    }
    x = 0;
    y = 0;
    width = null;
```

(continues on next page)

(continued from previous page)

```

    height = null;
}

//Rect's constructor has 2 parameters so the class has to be 'called'
//with 2 parameters
local rc = Rect(100,100);

```

After an instance is created, its properties can be set or fetched following the same rules that apply to tables. Methods cannot be set.

Instance members cannot be removed.

The class object that created a certain instance can be retrieved through the built-in function `instance.getclass()` (see *built-in functions*)

The operator `instanceof` tests if a class instance is an instance of a certain class.:

```

local rc = Rect(100,100);
if(rc instanceof ::Rect) {
    ::print("It's a rect");
}
else {
    ::print("It isn't a rect");
}

```

Note: Since Squirrel 3.x `instanceof` doesn't throw an exception if the left expression is not a class, it simply fails

3.9.3 Inheritance

Squirrel's classes support single inheritance by adding the keyword `extends`, followed by an expression, in the class declaration. The syntax for a derived class is the following:

```

class SuperFoo extends Foo {
    function DoSomething() {
        ::print("I'm doing something");
    }
}

```

When a derived class is declared, Squirrel first copies all base's members in the new class then proceeds with evaluating the rest of the declaration.

A derived class inherit all members and properties of it's base, if the derived class overrides a base function the base implementation is shadowed. It's possible to access a overridden method of the base class by fetching the method from through the 'base' keyword.

Here an example:

```

class Foo {
    function DoSomething() {
        ::print("I'm the base");
    }
};

class SuperFoo extends Foo {
    //overridden method
    function DoSomething() {

```

(continues on next page)

(continued from previous page)

```
//calls the base method
base.DoSomething();
::print("I'm doing something");
}
}
```

Same rule apply to the constructor. The constructor is a regular function (apart from being automatically invoked on construction).:

```
class BaseClass {
    constructor()
    {
        ::print("Base constructor\n");
    }
}

class ChildClass extends BaseClass {
    constructor()
    {
        base.constructor();
        ::print("Child constructor\n");
    }
}

local test = ChildClass();
```

The base class of a derived class can be retrieved through the built-in method `getbase()` .:

```
local thebaseclass = SuperFoo.getbase();
```

Note that because methods do not have special protection policies when calling methods of the same objects, a method of a base class that calls a method of the same class can end up calling a overridden method of the derived class.

A method of a base class can be explicitly invoked by a method of a derived class though the keyword `base` (as in `base.MyMethod()`).:

```
class Foo {
    function DoSomething() {
        ::print("I'm the base");
    }
    function DoIt()
    {
        DoSomething();
    }
};

class SuperFoo extends Foo {
    //overridden method
    function DoSomething() {
        ::print("I'm the derived");
    }
    function DoIt() {
        base.DoIt();
    }
}
```

(continues on next page)

(continued from previous page)

```
//creates a new instance of SuperFoo
local inst = SuperFoo();

//prints "I'm the derived"
inst.DoIt();
```

3.9.4 Metamethods

Class instances allow the customization of certain aspects of their semantics through metamethods. For C++ programmers: “metamethods behave roughly like overloaded operators”. The metamethods supported by classes are `_add`, `_sub`, `_mul`, `_div`, `_unm`, `_modulo`, `_set`, `_get`, `_typeof`, `_nexti`, `_cmp`, `_call`, `_delslot`, `_tostring`

Class objects instead support only 2 metamethods : `_newmember` and `_inherited`

the following example show how to create a class that implements the metamethod `_add`:

```
class Vector3 {
  constructor(...)
  {
    if(vargv.len() >= 3) {
      x = vargv[0];
      y = vargv[1];
      z = vargv[2];
    }
  }
  function _add(other)
  {
    return ::Vector3(x+other.x,y+other.y,z+other.z);
  }

  x = 0;
  y = 0;
  z = 0;
}

local v0 = Vector3(1,2,3)
local v1 = Vector3(11,12,13)
local v2 = v0 + v1;
::print(v2.x+", "+v2.y+", "+v2.z+"\n");
```

Since version 2.1, classes support 2 metamethods `_inherited` and `_newmember`. `_inherited` is invoked when a class inherits from the one that implements `_inherited`. `_newmember` is invoked for each member that is added to the class(at declaration time).

3.10 Constants & Enumerations

Squirrel allows to bind constant values to an identifier that will be evaluated compile-time. This is achieved through constants and Enumerations.

3.10.1 Constants

Constants bind a specific value to an identifier. Constants are similar to global values, except that they are evaluated compile time and their value cannot be changed.

constants values can only be integers, floats or string literals. No expression are allowed. are declared with the following syntax.:

```
const foobar = 100;
const floatbar = 1.2;
const stringbar = "I'm a constant string";
```

constants are always globally scoped, from the moment they are declared, any following code can reference them. Constants will shadow any global slot with the same name(the global slot will remain visible by using the :: syntax).:

```
local x = foobar * 2;
```

3.10.2 Enumerations

As Constants, Enumerations bind a specific value to a name. Enumerations are also evaluated at compile time and their value cannot be changed.

An enum declaration introduces a new enumeration into the program. Enumeration values can only be integers, floats or string literals. No expression are allowed.:

```
enum Stuff {
    first, //this will be 0
    second, //this will be 1
    third //this will be 2
}
```

or:

```
enum Stuff {
    first = 10
    second = "string"
    third = 1.2
}
```

An enum value is accessed in a manner that's similar to accessing a static class member. The name of the member must be qualified with the name of the enumeration, for example `Stuff.second` Enumerations will shadow any global slot with the same name(the global slot will remain visible by using the :: syntax).:

```
local x = Stuff.first * 2;
```

3.10.3 Implementation notes

Enumerations and Constants are a compile-time feature. Only integers, string and floats can be declared as const/enum; No expressions are allowed(because they would have to be evaluated compile time). When a const or an enum is declared, it is added compile time to the `consttable`. This table is stored in the VM shared state and is shared by the VM and all its threads. The `consttable` is a regular squirrel table; In the same way as the `roottable` it can be modified runtime. You can access the `consttable` through the built-in function `getconsttable()` and also change it through the built-in function `setconsttable()`

here some example:

```
//creates a constant
getconsttable()["something"] <- 10
//creates an enumeration
getconsttable()["somethingelse"] <- { a = "10", c = "20", d = "200"};
//deletes the constant
delete getconsttable()["something"]
//deletes the enumeration
delete getconsttable()["somethingelse"]
```

This system allows to procedurally declare constants and enumerations, it is also possible to assign any squirrel type to a constant/enumeration(function,classes etc...). However this will make serialization of a code chunk impossible.

3.11 Built-in Functions

3.11.1 Global Symbols

array(*size*[, *fill*])

creates and returns array of a specified size. If the optional parameter *fill* is specified its value will be used to fill the new array's slots. If the *fill* parameter is omitted, null is used instead.

assert(*exp*[, *message*])

throws an exception if *exp* is null or false. Throws "assertion failed" string by default, or *message* if specified.

print(*x*)

prints *x* to the standard output

error(*x*)

prints *x* in the standard error output

type(*obj*)

return the 'raw' type of an object without invoking the metamethod '_typeof'.

Default delegates

Except null and userdata every squirrel object has a default delegate containing a set of functions to manipulate and retrieve information from the object itself.

3.11.2 Integer

integer.tofloat()

convert the number to float and returns it

integer.tostring()

converts the number to string and returns it

integer.tointeger()

dummy function; returns the value of the integer.

integer.tochar()

returns a string containing a single character represented by the integer.

3.11.3 Float

float.tofloat()

returns the value of the float(dummy function)

float.tointeger()

converts the number to integer and returns it

float.tostring()

converts the number to string and returns it

`float.tochar()`

returns a string containing a single character represented by the integer part of the float.

3.11.4 Bool

`bool.tofloat()`

returns 1.0 for true 0.0 for false

`bool.tointeger()`

returns 1 for true 0 for false

`bool.tostring()`

returns “true” for true and “false” for false

3.11.5 String

`string.len()`

returns the string length

`string.tointeger([base])`

Converts the string to integer and returns it. An optional parameter base can be specified—if a base is not specified, it defaults to base 10.

`string.tofloat()`

converts the string to float and returns it

`string.tostring()`

returns the string (really, a dummy function)

`string.slice(start[, end])`

returns a section of the string as new string. Copies from start to the end (not included). If start is negative the index is calculated as length + start, if end is negative the index is calculated as length + end. If end is omitted end is equal to the string length.

`string.find(substr[, startidx])`

Searches a sub string (substr) starting from the index startidx and returns the index of its first occurrence. If startidx is omitted the search operation starts from the beginning of the string. The function returns null if substr is not found.

`string.tolower()`

returns a lowercase copy of the string.

`string.toupper()`

returns an uppercase copy of the string.

3.11.6 Table

table.len()

returns the number of slots contained in a table

table.tostring()

Tries to invoke the `_tostring` metaclass. If that fails, it returns “(table : pointer)”.

table.clear()

removes all the slots from the table. Returns table itself.

table.filter(*func(key, val)*)

Creates a new table with all values that pass the test implemented by the provided function. In detail, it creates a new table, invokes the specified function for each key-value pair in the original table; if the function returns ‘true’, then the value is added to the newly created table at the same key.

table.keys()

returns an array containing all the keys of the table slots.

table.values()

returns an array containing all the values of the table slots.

3.11.7 Array

array.len()

returns the length of the array

array.append(*val*)

appends the value ‘val’ at the end of the array. Returns array itself.

array.push(*val*)

appends the value ‘val’ at the end of the array. Returns array itself.

array.extend(*array*)

Extends the array by appending all the items in the given array. Returns array itself.

array.pop()

removes a value from the back of the array and returns it.

array.top()

returns the value of the array with the higher index

array.insert(*idx, val*)

inserts the value ‘val’ at the position ‘idx’ in the array. Returns array itself.

array.remove(*idx*)

removes the value at the position ‘idx’ in the array and returns its value.

array.resize(*size[, fill]*)

Resizes the array. If the optional parameter ‘fill’ is specified, its value will be used to fill the new array’s slots when the size specified is bigger than the previous size. If the fill parameter is omitted, null is used instead. Returns array itself.

`array.sort([compare_func])`

Sorts the array in-place. A custom compare function can be optionally passed. The function prototype as to be the following.:

```
function custom_compare(a,b)
{
    if(a>b) return 1
    else if(a<b) return -1
    return 0;
}
```

a more compact version of a custom compare can be written using a lambda expression and the operator `<=>`

```
arr.sort(@(a,b) a <=> b);
```

Returns array itself.

`array.reverse()`

reverse the elements of the array in place. Returns array itself.

`array.slice(start[, end])`

Returns a section of the array as new array. Copies from start to the end (not included). If start is negative the index is calculated as `length + start`, if end is negative the index is calculated as `length + end`. If end is omitted end is equal to the array length.

`array.toString()`

returns the string "(array : pointer)".

`array.clear()`

removes all the items from the array

`array.map(func(item_value, [item_index], [array_ref]))`

Creates a new array of the same size. For each element in the original array invokes the function 'func' and assigns the return value of the function to the corresponding element of the newly created array. Provided func can accept up to 3 arguments: array item value (required), array item index (optional), reference to array itself (optional).

`array.apply(func([item_value], [item_index], [array_ref]))`

for each element in the array invokes the function 'func' and replace the original value of the element with the return value of the function.

`array.reduce(func(prevval, curval)[, initializer])`

Reduces an array to a single value. For each element in the array invokes the function 'func' passing the initial value (or value from the previous callback call) and the value of the current element. The return value of the function is then used as 'prevval' for the next element. If the optional initializer is present, it is placed before the items of the array in the calculation, and serves as a default when the sequence is empty. If initializer is not given then for sequence contains only one item, `reduce()` returns the first item, and for empty sequence returns null.

Given an sequence with 2 or more elements (including initializer) calls the function with the first two elements as the parameters, gets that result, then calls the function with that result and the third element, gets that result, calls the function with that result and the fourth parameter and so on until all element have been processed. Finally, returns the return value of the last invocation of func.

`array.filter(func(index, val))`

Creates a new array with all elements that pass the test implemented by the provided function. In detail, it creates a new array, for each element in the original array invokes the specified function passing the index of the element and it's value; if the function returns 'true', then the value of the corresponding element is added on the newly created array.

`array.find(value)`

Performs a linear search for the value in the array. Returns the index of the value if it was found null otherwise.

3.11.8 Function

`function.call(_this, args...)`

calls the function with the specified environment object('this') and parameters

`function.pcall(_this, args...)`

calls the function with the specified environment object('this') and parameters, this function will not invoke the error callback in case of failure(pcall stays for 'protected call')

3.11.9 Class

`class.instance()`

returns a new instance of the class. this function does not invoke the instance constructor. The constructor must be explicitly called (eg. `class_inst.constructor(class_inst)`).

3.11.10 Class Instance

`instance.getclass()`

returns the class that created the instance.

EBLINK SCRIPTING

4.1 Introduction

EBlink uses [Squirrel](#) as its scripting language. Scripts drive the device-specific behaviour: they set up the memory map, implement flash erase and write algorithms, handle custom resets, and can interact with the GDB monitor interface.

A C/C++ background is assumed throughout this chapter. Squirrel syntax is covered in the language reference; this chapter focuses on what EBLink exposes to scripts and how scripts integrate with the EBLink runtime.

4.1.1 How scripts are loaded

Scripts are specified on the command line with `-S`. Multiple `-S` options are allowed; each file is loaded in order into the same Squirrel VM so all scripts share one global namespace and see each other's definitions:

```
EBlink -I stlink -S stm32.script -G
EBlink -I stlink -S mydevice -S myReset.scr -G
```

Script files are searched in the paths added with `-P`. Multiple `-P` options accumulate search paths:

```
EBlink -I stlink -P C:\eblink\scripts -S stm32 -G
```

4.1.2 CLI defines

Key/value pairs can be passed to scripts at startup with `-D`. Multiple `-D` options are allowed:

```
EBlink -I stlink -S stm32 -D FLASH_SIZE=1024 -D RAM_SIZE=16 -G
```

Inside the script these are available as global integer constants:

```
printf("Flash speed: %d KHz\n", FLASH_SPEED);
```

4.1.3 API objects

`InterfAPI`, `DeviceAPI`, `TargetAPI`, and `ServicesAPI` are classes registered by EBLink. A script instantiates them itself, typically at the top of the main script as globals using the `<-` operator:

```
intrfApi <- InterfAPI()
targetApi <- TargetAPI()
```

These globals are then accessible to all scripts loaded via `require` or `include`. `DeviceAPI()` is usually called inline where the memory map needs to be set.

4.1.4 Minimal example

The smallest useful top-level script: instantiate the API objects, set the memory map, and define flash entry points. Flash programming requires device-specific register sequences — the body of each function is device-specific and omitted here:

```
intrfApi <- InterfAPI()
targetApi <- TargetAPI()

function ebMain()
{
    intrfApi.setSpeed(4000);
```

(continues on next page)

(continued from previous page)

```
DeviceAPI().memmap( @"<?xml version=\"1.0\"?>
<memory-map>
  <memory type=\"ram\" start=\"0x20000000\" length=\"0x8000\"/>
  <memory type=\"flash\" start=\"0x08000000\" length=\"0x40000\">
    <property name=\"pagesize\">0x400</property>
  </memory>
</memory-map>" );
}

function ebFlashStart()
{
  // Unlock flash, reset target to known state.
  return ERROR_OK;
}

function ebFlashErase(sector, address)
{
  // Device-specific sector erase via intrfApi.write(...).
  return ERROR_OK;
}

function ebFlashWrite(sector, address, buffer)
{
  // Device-specific flash write via intrfApi.writeBuf(...).
  return ERROR_OK;
}

function ebFlashDone()
{
  // Re-lock flash.
  return ERROR_OK;
}
```

4.2 Error Handling

EBlink uses Squirrel exceptions for all error signalling. Understanding the convention is essential because every API call can throw.

4.2.1 Exception types

API methods throw one of two exception types:

Integer exception — a negative `ERROR_XXX` constant. Thrown by hardware operations when the probe reports an error (target lost, flash failure, etc.):

```
try {
    local value = intrfApi.read(0x200000000);
}
catch(e) {
    if(isInteger(e))
        fprintf("Hardware error %d: %s\n", e, errorToStr(e));
}
```

String exception — a human-readable message. Thrown for usage errors: wrong argument type, buffer overflow, NULL instance, etc.:

```
try {
    local buf = EBbuffer(64);
    buf.get(200);           // offset beyond buffer
}
catch(e) {
    if(isString(e))
        fprintf("Script error: %s\n", e);
}
```

4.2.2 Return values

Functions that succeed return `ERROR_OK` (0) or a positive value (e.g. byte count). A negative return value is always an `ERROR_XXX` code. The convention is that you either throw on failure or return the code — never both:

```
function ebFlashWrite(sector, address, buffer)
{
    try {
        intrfApi.writeBuf(buffer, address);
    }
    catch(e) {
        if(isString(e)) { fprintf(e); return ERROR_NOTIFIED; }
        if(isInteger(e)) { return e; }
    }
    return ERROR_OK;
}
```

4.2.3 Propagating errors

Use `ERROR_NOTIFIED` (-1) when you have already printed an error message and want EBLink to abort without printing a second one:

```
errorf("Flash unlock failed at 0x%08X\n", addr);
return ERROR_NOTIFIED;
```

For errors the script did not handle, re-throw so EBLink can report them:

```
catch(e) {
    if(catchString(e)) return ERROR_NOTIFIED; // string: already printed
    throw e;                               // integer: let EBLink handle
}
```

4.2.4 Helper functions

catchString(value[, throwValue])

If *value* is a string exception, prints it as an error and returns `true`. If *throwValue* is supplied and *value* is a string, throws *throwValue* after printing. Returns `false` if *value* is not a string.

Typical use in a catch block:

```
catch(e) {
    if(catchString(e)) return ERROR_NOTIFIED;
    throw e;
}
```

catchThrowStr(value)

If *value* is a string, re-throws it as a string exception. Has no effect when *value* is not a string. Useful to propagate string errors up through nested try/catch blocks.

errorToStr(code)

Returns a short description string for an `ERROR_xxx` integer code. Returns an empty string for unknown codes.

Example:

```
errorf("Error %d: %s\n", e, errorToStr(e));
```

4.2.5 User-defined error codes

`ERROR_USER` (-1000) is the boundary reserved for script errors. All built-in `ERROR_xxx` codes are between -1 and -999. Scripts must define their own codes **below** `ERROR_USER` to avoid collisions:

```
const FLASH_UNLOCK_FAIL = ERROR_USER - 1; // -1001
const FLASH_TIMEOUT     = ERROR_USER - 2; // -1002
const FLASH_VERIFY_FAIL = ERROR_USER - 3; // -1003
```

These constants behave exactly like built-in error codes — they can be thrown, returned, or passed to `errorToStr()`. Because `errorToStr()` does not know about script-defined codes it returns an empty string for them; print your own message before returning or throwing:

```
function unlockFlash()
{
    if(intrfApi.read(FLASH_CR) & FLASH_LOCKED) {
```

(continues on next page)

(continued from previous page)

```
    intrfApi.write(FLASH_KEYR, KEY1);
    intrfApi.write(FLASH_KEYR, KEY2);
    if(intrfApi.read(FLASH_CR) & FLASH_LOCKED) {
        errorf("Flash unlock failed\n");
        return FLASH_UNLOCK_FAIL;
    }
}
return ERROR_OK;
}
```

Note: Never define script error codes between -1 and -999 — that range belongs to EBlink built-ins. Always subtract from ERROR_USER.

4.3 Logging & Output

EBlink routes script output through its verbosity system. Each output function maps to a verbosity level; output is only shown when the corresponding level is active. The verbosity is set with the `-v` command-line flag.

4.3.1 Output functions

All output functions accept a `printf`-style format string followed by optional arguments. They do not return a value.

printf(fmt, ...)

Default script output. Visible at `-v4` and above (`SCRIPT_INFO`).

userf(fmt, ...)

User-facing output. Visible at `-v2` and above (`USER_INFO`). For messages relevant to the end user.

noticef(fmt, ...)

Notice-level output. Visible at `-v3` and above (`NOTICE_INFO`).

warnf(fmt, ...)

Warning output. Visible at `-v6` and above (`WARN_INFO`).

errorf(fmt, ...)

Error output. Always visible regardless of verbosity level.

debugf(fmt, ...)

Debug output. Visible at `-v7` and above (`DEBUG_INFO`). For detailed internal state during script development.

tracef(fmt, ...)

Trace output. Visible at `-v8` (`TRACE_INFO`). For fine-grained step-by-step tracing.

yesNof(default, fmt, ...)

Presents an interactive yes/no question. `default` is returned when no interactive console is available: 1 for yes, 0 for no. Returns the user's choice or `default`:

```
local answer = yesNof(0, "Erase entire chip? [y/n] ");
if(answer)
    device_chip_erase();
```

4.3.2 Verbosity levels

Flag	Constant	Functions visible
<code>-v2</code>	<code>USER_INFO</code>	<code>userf</code>
<code>-v3</code>	<code>NOTICE_INFO</code>	<code>noticef</code>
<code>-v4</code>	<code>SCRIPT_INFO</code>	<code>printf</code>
<code>-v6</code>	<code>WARN_INFO</code>	<code>warnf</code>
<code>-v7</code>	<code>DEBUG_INFO</code>	<code>debugf</code>, <code>brkPt</code>
<code>-v8</code>	<code>TRACE_INFO</code>	<code>tracef</code>, <code>line step</code>

`errorf` is always shown. `-v1` shows only the EBLink header.

4.3.3 Format strings

Standard C `printf` conventions: `%d`, `%u`, `%x`, `%X`, `%f`, `%s`, `%c`. Width and precision modifiers are supported. Use `%08X` for addresses (no `%p`):

```
printf("Sector %d at 0x%08X (%d bytes)\n", sector, address, buffer.getSize());
```

4.4 Global Functions

These functions are available in all EBLink scripts without any import or declaration.

4.4.1 Script loading

include(*filename*)

Loads and executes a Squirrel script file. Searches the configured script paths. Does nothing and returns `ERROR_OK` if the file is not found — use `require` when the file must exist.

require(*filename*)

Loads and executes a Squirrel script file. Throws a string exception if the file cannot be found or loaded. Use for dependencies that must be present.

4.4.2 Type checking

isString(*value*)

Returns `true` if *value* is a string.

isInteger(*value*)

Returns `true` if *value* is an integer.

isnull(*instance*)

Returns `true` if *instance* is a null object instance. Used to check whether an API object reference is valid.

isScriptObject(*name*)

Returns `true` if a global function or object with the given name exists in the script VM. Useful before calling an optional entry point:

```
if(isScriptObject("myCustomInit"))
    myCustomInit();
```

4.4.3 Data conversion

strToNumber(*str*)

Parses a decimal or hexadecimal (`0x` prefix) string and returns the value as an integer. Returns `0` for invalid input.

Example:

```
local addr = strToNumber("0x08004000");
```

Integer2Float(*value*)

Reinterprets the bit pattern of a 32-bit integer as an IEEE 754 float. Equivalent to a C `*(float*)&value` cast. Useful when a register or memory location holds a float value.

Float2Integer(*value*)

Reinterprets the bit pattern of a float as a 32-bit integer. Equivalent to a C `*(uint32_t*)&value` cast.

4.4.4 EBLink information

cliSwitch(*switch*)

Returns the value string of a CLI option. Recognised switches are "-I" (interface), "-T" (target), "-F" (flash), "-G" (GDB). Returns an empty string if the option was not supplied. When multiple -G options are given, only the first one is returned:

```
local iface_opt = cliSwitch("-I");
if(iface_opt == "")
    errorf("No interface specified\n");
```

Also used in `ebServicesSetup()` to pass the original CLI service options to `ServicesAPI().addService()` — see *ServicesAPI*.

4.4.5 String utilities

astrsep(*array*, *string*, *separator*)

Splits *string* on each occurrence of *separator* and appends the resulting tokens to *array*. Returns the number of tokens added.

Example:

```
local parts = [];
astrsep(parts, "one:two:three", ":");
// parts == ["one", "two", "three"]
```

amregex(*array*, *string*, *pattern*)

Applies the regular expression *pattern* to *string*. Appends all capture groups (or the full match if there are no groups) to *array*. Returns the number of matches appended, or 0 if no match.

Example:

```
local m = [];
if(amregex(m, "REV=1.4", "REV=([0-9.]+)"))
    printf("Revision: %s\n", m[0]);
```

4.4.6 System utilities

Sleep(*ms*)

Suspends execution for *ms* milliseconds.

GetTickCount()

Returns the number of milliseconds elapsed since EBLink started. Useful for implementing timeouts:

```
local t0 = GetTickCount();
while(GetTickCount() - t0 < 500) {
    if(target.getState() == TARGET_HALTED) break;
    Sleep(10);
}
```

AnimateCursor()

Starts a spinning cursor animation on the console. Call before a long operation.

AnimateDone()

Stops the cursor animation started by `AnimateCursor()`.

4.4.7 Math

Each function takes two values of the appropriate type and returns the smaller or larger of the two.

FL_MIN(*a*, *b*)

FL_MAX(*a*, *b*)

Float minimum / maximum.

UN_MIN(*a*, *b*)

UN_MAX(*a*, *b*)

Unsigned integer minimum / maximum.

IN_MIN(*a*, *b*)

IN_MAX(*a*, *b*)

Integer minimum / maximum.

4.5 Constants

All constants listed here are available in every EBLink script without declaration.

4.5.1 VERSION

VERSION is a float containing the EBLink API version:

```
if(VERSION < 3.0)
    errorf("This script requires EBLink API 3.0 or newer\n");
```

4.5.2 Target state

Returned by `TargetAPI.getState()`.

Constant	Value	Meaning
TARGET_UNKNOWN	0	State cannot be determined
TARGET_RUNNING	1	CPU is executing code
TARGET_HALTED	2	CPU is halted (debug state)
TARGET_RESET	3	CPU is held in reset

4.5.3 Reset types

Passed to `TargetAPI.reset()`.

Constant	Value	Effect
SYSTEM_RESET	0	Full system reset via AIRCR.SYSRESETREQ
CORE_RESET	1	Core-only reset via AIRCR.VECTRESET
JTAG_RESET	2	Hardware reset via the nRESET pin
USER_RESET	3	Custom reset — calls <code>ebResetUser()</code>

4.5.4 Error codes

All negative integers thrown by API methods correspond to one of these constants. Scripts can compare a caught integer exception against them:

```
catch(e) {
    if(e == ERROR_TARGET_LOST) { errorf("Target disconnected\n"); return e; }
    throw e;
}
```

User-defined error codes must be more negative than `ERROR_USER`:

```
const MY_UNLOCK_FAIL = (ERROR_USER - 1); // -1001
```

Constant	Value	Description
ERROR_OK	0	No error
ERROR_NOTIFIED	-1	Error already reported, abort silently
ERROR_SYNTAX	-2	Script syntax error
ERROR_OUT_MEMM	-3	Out of memory
ERROR_TARGET_LOST	-4	Target connection lost
ERROR_SOCKET_CLOSE	-5	GDB socket closed
ERROR_SOCKET_ERR	-6	GDB socket error
ERROR_NO_DEVICE	-7	No device found
ERROR_OUT_OF_RANGE	-8	Value out of range
INTERFACE_NOT_SUPPORTED	-20	Feature not supported by this interface
INTERFACE_WAIT	-21	Interface speed too high
INTERFACE_ERROR	-22	Interface communication error
ERROR_INTERNAL	-23	Internal EBlind error
ERROR_NOT_FOUND	-100	Requested item not found
ERROR_UNLOCK	-300	Flash unlock failed
ERROR_VOLTAGE	-301	Target voltage out of range
ERROR_FLASH	-302	Flash operation failed
ERROR_TIMEOUT	-303	Operation timed out
ERROR_USER	-1000	Base for user-defined error codes

INTERFACE_WAIT (-21) usually means the probe speed set with `iface.setSpeed()` is too high for the target. Try reducing it or use the `-D FLASH_SPEED=<KHz>` CLI option.

4.6 EBLink Interaction

EBlink calls specific functions in the script at defined points in its operation. Functions not defined in the script are silently skipped, except where marked mandatory.

4.6.1 Overview

Function	Required	Called when
ebMain()	mandatory	All scripts loaded, CLI defines set
ebFlashStart()	optional	Flash operation is about to begin
ebFlashErase(sector_nr, sector_addr)	optional	A sector must be erased before writing
ebFlashWrite(sector_nr, addr, buf)	mandatory	A sector must be written
ebFlashDone()	optional	All flash sectors processed
ebFlashEraseChip()	optional	Full chip erase requested (-F erase)
ebResetUser()	optional*	A USER_RESET is requested
ebResetPost(reset_type)	optional	After every reset, regardless of type
ebServicesSetup()	optional	EBlink service initialisation
ebGDBqRcmd(cmd, length, buffer)	optional	GDB monitor command received

* Mandatory when the script uses USER_RESET as its reset strategy.

4.6.2 Call order

```

Script loaded
|
| v
ebMain()                <- mandatory: set memmap, initialise device
|
| +--- flash operation requested
| |
| | v
| | ebFlashStart()      <- optional: prepare hardware for flash
| |
| | +-----+
| | | for each modified sector:
| | | ebFlashErase(nr, addr) <- optional
| | | ebFlashWrite(nr, addr, buf) <- mandatory
| | |
| | +-----+
| |
| | ebFlashDone()      <- optional: clean up after flash
| |
| | +--- chip erase requested
| | |
| | | ebFlashEraseChip() <- optional (falls back to sector loop)
| | |
| | +--- reset requested
| | |
| | | ebResetUser()      <- mandatory when USER_RESET type used
| | | ebResetPost(type)  <- optional: called after every reset

```

ebServicesSetup() is called once during service initialisation, independent of the flash or reset flow. ebGDBqRcmd() is called on every incoming GDB monitor command, independent of the other flows.

4.6.3 Reference

ebMain()

mandatory

- Called once after all scripts are loaded and all -D defines are set.
- Must call DeviceAPI().memmap() — EBLink aborts if no memory map is set.
- Typical use: detect device, set interface speed, configure flash parameters.

Returns: nothing required.

```
intrfApi <- InterfAPI()
targetApi <- TargetAPI()

function ebMain()
{
    intrfApi.setSpeed(4000);

    DeviceAPI().memmap( @@"<?xml version=\"1.0\"?>
<memory-map>
  <memory type=\"flash\" start=\"0x08000000\" length=\"0x80000\">
    <property name=\"pagesize\">0x800</property>
  </memory>
  <memory type=\"ram\" start=\"0x20000000\" length=\"0x20000\"/>
</memory-map>" );
}
```

ebFlashStart()

optional

- Called once before the first sector is touched.
- Typical use: unlock flash, reset target to known state, set interface speed.

Returns: ERROR_OK or a negative error code.

ebFlashErase(sector_nr, sector_addr)

optional

- sector_nr — zero-based sector index from the memory map.
- sector_addr — start address of the sector.
- If not defined, EBLink performs no explicit sector erase.

Returns: ERROR_OK or a negative error code.

ebFlashWrite(sector_nr, sector_addr, buffer)

mandatory when flashing

- sector_nr — zero-based sector index from the memory map.
- sector_addr — start address of the sector in target memory.
- buffer — *EBbuffer* containing the sector data, trimmed to the last non-erased byte.
- Called for every sector that needs to be written, after erasure.

Returns: ERROR_OK or a negative error code.

```
function ebFlashWrite(sector, address, buffer)
{
    try {
        intrfApi.writeBuf(buffer, address);
```

(continues on next page)

(continued from previous page)

```

    }
    catch(e) {
        if(catchString(e)) return ERROR_NOTIFIED;
        return e;
    }
    return ERROR_OK;
}

```

ebFlashDone()*optional*

- Called once after all sectors have been processed, success or failure.
- Typical use: re-lock flash, reset the device.

Returns: ERROR_OK or a negative error code.

ebFlashEraseChip()*optional*

- Called when a full chip erase is requested (-F erase option).
- Runs standalone — ebFlashStart and ebFlashDone are **not** called around it.
- If not defined, EBLink falls back to calling ebFlashStart, ebFlashErase for every sector, and ebFlashDone.

Returns: ERROR_OK or a negative error code.

ebResetUser()*mandatory when USER_RESET type is used*

- Implement a custom reset sequence — for example via a peripheral register when the nRESET pin is not available.

Returns: nothing required.

```

function ebResetUser()
{
    intrfApi.write(0xE000ED0C, 0x05FA0004); // SYSRESETREQ via AIRCR
    Sleep(50);
}

```

ebResetPost(reset_type)*optional*

- reset_type — one of the xxx_RESET constants (see *Constants*).
- Called after every reset, regardless of type.
- Typical use: set stack pointer and PC from the vector table, wait for the device to come out of reset.

Returns: ERROR_OK or a negative error code.

```

function ebResetPost(type)
{
    if(type != USER_RESET) {
        targetApi.writeReg("SP", intrfApi.read(0x08000000));
        targetApi.writeReg("PC", intrfApi.read(0x08000004));
    }
    return ERROR_OK;
}

```

ebServicesSetup()*optional*

- Called during EBLink service initialisation.
- If not defined, EBLink starts services as specified on the CLI.
- As soon as one `ServicesAPI().addService()` call is made, EBLink no longer starts any services automatically — the script is responsible for starting all services.
- Use `cliSwitch("-G")` to retrieve the CLI GDB option and pass it to `addService()` if needed.

Returns: `ERROR_OK` or a negative error code.

ebGDBqRcmd(cmd, length, buffer)*optional*

- Called when a GDB monitor command arrives, before EBLink's own built-in monitor handling.
- `cmd` — the monitor command string.
- `length` — length of the command string in bytes.
- `buffer` — an *EBbuffer* the script fills with the response text to send back to GDB.
- If the script handles the command: fill `buffer` and return `ERROR_OK`. EBLink sends the buffer contents back to GDB and skips built-in handling.
- If the script does not handle the command: return a negative error code or leave `buffer` empty. EBLink then falls through to its own monitor command dispatcher.

Returns: `ERROR_OK` (command handled) or a negative error code (pass through).

The two examples below show the same `ReadMemAP` command implemented with two different parsing strategies.

Parsing with `amregex` — a single regex captures all arguments at once:

```
function ebGDBqRcmd(packet, length, ebBuffer)
{
    try {
        local matches = []
        // ReadMemAP <APBaseAddr> <Addr> <NumItems> <Flags>
        if(amregex(matches, packet,
            @"ReadMemAP\s+(0x[0-9a-fA-F]+|[0-9]+)\s+(0x[0-9a-fA-F]+|[0-9]+)" +
            @"\s+(0x[0-9a-fA-F]+|[0-9]+)\s+(0x[0-9a-fA-F]+|[0-9]+)" > 0)
        {
            local APbase = strToNumber(matches[1])
            local addr    = strToNumber(matches[2])
            local num     = strToNumber(matches[3])
            local flags   = strToNumber(matches[4])

            local byteLen = (flags == 0 ? 4 : flags)
            local width   = (flags == 0 ? -1 : byteLen * 8)
            local fspec    = format("0x%%0%dX", byteLen * 2)

            local eb = EBbuffer()
            InterfAPI().readBuf(eb, addr, num * byteLen, width)
            if(width > 0) eb.access(width)

            local outStr = "O.K.:"
            for(local i = 0; i < num; i++) {
                outStr += format(fspec, eb[i])
                if(i < num - 1) outStr += ","
            }
        }
    }
}
```

(continues on next page)

(continued from previous page)

```

        ebBuffer.format(outStr + "\n")
    }
    return ERROR_OK
}
catch(e) {
    if(catchString(e)) return ERROR_NOTIFIED
    return ERROR_NOTIFIED
}
}

```

Parsing with astrsep — tokenises on spaces, then checks token by name:

```

function ebGDBqRcmd(packet, length, ebBuffer)
{
    try {
        local tokens = []
        astrsep(tokens, packet, " ")

        // ReadMemAP <APBaseAddr> <Addr> <NumItems> <Flags>
        if(tokens[0] == "ReadMemAP") {
            if(tokens.len() < 5) return ERROR_OK

            local APbase = strToNumber(tokens[1])
            local addr = strToNumber(tokens[2])
            local num = strToNumber(tokens[3])
            local flags = strToNumber(tokens[4])

            local byteLen = (flags == 0 ? 4 : flags)
            local width = (flags == 0 ? -1 : byteLen * 8)
            local fspec = format("0x%%0dX", byteLen * 2)

            local eb = EBbuffer()
            InterfAPI().readBuf(eb, addr, num * byteLen, width)
            if(width > 0) eb.access(width)

            local outStr = "O.K.:"
            for(local i = 0; i < num; i++) {
                outStr += format(fspec, eb[i])
                if(i < num - 1) outStr += ","
            }
            ebBuffer.format(outStr + "\n")
        }
        return ERROR_OK
    }
    catch(e) {
        if(catchString(e)) return ERROR_NOTIFIED
        return ERROR_NOTIFIED
    }
}

```

Use `amregex` when the command syntax is complex or needs strict validation. Use `astrsep` when the command is simple whitespace-delimited tokens.

4.7 EBuffer

EBuffer is a general-purpose binary data buffer. Unlike the other API objects it can be created freely by scripts. It is the standard way to pass blocks of data to and from hardware interface functions.

An EBuffer has a fixed size once allocated, a configurable access width (8, 16, or 32 bits) that governs how the `[]` index operator works, and an optional trim value for alignment padding.

4.7.1 Construction

EBuffer(*[size_or_other]*)

Creates an EBuffer. Three forms:

- **EBuffer()** — empty, unallocated. Use **allocate()** before access, or pass directly to **readBuf()** which allocates internally.
- **EBuffer(size)** — allocates *size* bytes, zero-initialised.
- **EBuffer(other)** — copies another EBuffer. Access width resets to 32 bits.

4.7.2 Memory access

EBuffer.get(*offset* [, *width*])

Returns the value at byte *offset*. *width* is 8, 16, or 32 bits; defaults to 32. Throws a string exception if the access would exceed the buffer.

EBuffer.set(*offset*, *value* [, *width*])

Writes *value* at byte *offset*. *width* is 8, 16, or 32 bits; defaults to 32. Throws a string exception if the access would exceed the buffer.

4.7.3 Index operator

buf[index] reads or writes one element using the current access width set by **access()**. The byte offset is *index* * (**accessWidth** / 8):

```
local buf = EBuffer(64);
buf.access(8);           // set 8-bit access
buf[0] = 0xDE;
buf[1] = 0xAD;
printf("0x%02X 0x%02X\n", buf[0], buf[1]);
```

4.7.4 Buffer management

EBuffer.allocate(*size*)

Allocates *size* bytes for an unallocated buffer. Throws if the buffer is already allocated or is a reference buffer. The buffer is zero-initialised.

EBuffer.getSize()

Returns the buffer size in bytes.

EBuffer.access(*[width]*)

Sets the access width used by the `[]` operator to *width* (8, 16, or 32). Passing `0` queries the current width without changing it. Returns the previous width.

Default access width is 32 bits.

EBbuffer.format(*fmt*, ...)

Fills the buffer with the result of a `printf`-style format operation, replacing any existing content. The buffer is re-allocated to fit the formatted string exactly. Returns the number of bytes written.

Useful for building XML or text payloads to write to target memory:

```
local buf = EBbuffer();
buf.format("BOOT=%d\n", boot_mode);
intrfApi.writeBuf(buf, 0x20001000);
```

4.7.5 Trimming

Trim is used with reference buffers (obtained from `readBuf`) to pad the logical size to an alignment boundary without allocating extra memory.

EBbuffer.trim(*boundary*)

Sets the trim so that `trimmedSize()` is rounded up to a multiple of *boundary* bytes. Only valid for reference buffers (those filled by `readBuf`). Pass `0` to clear the trim.

EBbuffer.getTrim()

Returns the current trim padding in bytes.

4.7.6 Operators

buf1 + buf2

Concatenates two buffers and returns a new **EBbuffer** containing the data of `buf1` followed by `buf2`.

buf1 <=> buf2

Compares the contents of two buffers byte by byte. Returns `0` if equal, `-1` if `buf1` is less than `buf2`, `1` if greater. Used by the Squirrel `==` and comparison operators.

4.7.7 Example

Reading a flash sector and comparing it to a file:

```
local sector = EBbuffer();
intrfApi.readBuf(sector, 0x08000000, 2048);

local expected = EBbuffer();
// ... fill expected from file ...

if((sector <=> expected) == 0)
    printf("Sector matches\n");
else
    printf("Sector differs\n");
```

4.8 InterfAPI

InterfAPI is a class registered by EBLink that represents the physical debug probe. Scripts instantiate it once at global scope and share the instance across all included files:

```
intrfApi <- InterfAPI()
```

All methods that communicate with hardware throw an integer **ERROR_XXX** exception on failure.

4.8.1 Probe information

InterfAPI.getName()

Returns the interface name string (e.g. "stlink").

InterfAPI.getTapId()

Returns the JTAG TAP ID / SWD target ID as a 32-bit integer. Returns 0 if the read fails.

InterfAPI.getSpeed()

Returns the current probe communication speed in KHz.

InterfAPI.setSpeed(speed)

Sets the probe speed to *speed* KHz. Throws on error:

```
intrfApi.setSpeed(4000);
```

InterfAPI.targetVoltage()

Returns the measured target VCC voltage as a float (volts).

InterfAPI.resetCtrl(state)

Asserts (*state* = **true**) or deasserts (*state* = **false**) the hardware nRESET line. Throws on error.

4.8.2 DAP access

These functions provide direct access to the ARM Debug Access Port (DAP) registers, bypassing the normal memory interface. Useful for reading the IDCODE register or configuring AP/DP registers during device identification.

InterfAPI.setDapAp(ap)

Selects the active Access Port (0–127). Returns the previously selected AP number. Throws on error.

InterfAPI.getDapAp()

Returns the currently selected Access Port number.

InterfAPI.readDap(ap, addr)

Reads a 32-bit value from DAP register *addr* on access port *ap*. Throws on error.

InterfAPI.writeDap(ap, addr, value)

Writes a 32-bit *value* to DAP register *addr* on access port *ap*. Throws on error.

4.8.3 Memory read

InterfAPI.read(*addr*[, *width*])

Reads a single value from target address *addr*. *width* is 8, 16, or 32 bits. When omitted (or -1), uses the probe's native 32-bit debug access path. Returns the value as an integer. Throws on error:

```
local cpuid = intrfApi.read(0xE000ED00);
printf("CPUID: 0x%08X\n", cpuid);
```

InterfAPI.readBuf(*buffer*, *addr*, *length*[, *width*])

Reads *length* bytes from target address *addr* into *buffer* (an `EBbuffer()`). The buffer is (re-)allocated to *length* bytes internally — no prior allocation is needed. *width* controls bus transfer width (8, 16, or 32); omit or pass -1 for the default. Returns *length* on success. Throws on error:

```
local buf = EBbuffer();
intrfApi.readBuf(buf, 0x08000000, 1024);
```

4.8.4 Memory write

InterfAPI.write(*addr*, *value*[, *width*])

Writes a single value to target address *addr*. *width* is 8, 16, or 32 bits; defaults to 32-bit debug access when omitted. Throws on error.

InterfAPI.writeBuf(*buffer*, *addr*[, *width*[, *offset*, *length*]])

Writes the contents of *buffer* to target address *addr*.

Optional parameters:

- *width* — bus transfer width: 8, 16, or 32. Omit or -1 for default.
- *offset* — byte offset into *buffer* to start from (must be paired with *length*).
- *length* — number of bytes to write (required when *offset* is given).

Throws on error:

```
// Write entire buffer
intrfApi.writeBuf(buf, 0x08000000);

// Write 256 bytes starting at offset 512
intrfApi.writeBuf(buf, 0x08001000, -1, 512, 256);
```

4.8.5 File loading

InterfAPI.loadBin(*filename*, *addr*)

Reads a binary file from disk and writes it to target memory starting at *addr*. EBlink searches the configured script paths for the file. Returns the number of bytes written. Throws an integer exception (via `ERROR_NOTIFIED`) if the file cannot be opened.

InterfAPI.loadString(*addr*, *str*, *len*)

Writes the raw bytes of *str* (up to *len* bytes) to target address *addr*. Returns *len*. Throws on hardware error. Useful for writing null-terminated strings or packed data to target RAM:

```
intrfApi.loadString(0x20000000, "BOOT", 4);
```

4.9 DeviceAPI

DeviceAPI is a class registered by EBLink that manages the device memory map and the flash sector cache. A new instance is created where needed:

```
DeviceAPI().memmap( ... )
```

The memory map must be set in `ebMain()`; without it EBLink cannot operate.

4.9.1 Memory map

DeviceAPI.**memmap**(*xml*)

Sets the device memory map from an XML string. The XML format is the GDB memory-map DTD. Must be called in `ebMain()`. Returns `ERROR_OK` on success or a negative error code if the XML could not be parsed.

The `blocksize` property inside a flash region defines the sector size used for the flash cache:

```
DeviceAPI().memmap( @"<?xml version=\"1.0\"?>
    <!DOCTYPE memory-map PUBLIC \"+//IDN gnu.org//DTD GDB Memory Map V1.0//EN\"
        \"http://sourceware.org/gdb/gdb-memory-map.dtd\">
    <memory-map>
        <memory type=\"flash\" start=\"0x08000000\" length=\"0x100000\">
            <property name=\"blocksize\">0x800</property>
        </memory>
        <memory type=\"ram\" start=\"0x20000000\" length=\"0x200000\"/>
    </memory-map>" );
```

Multiple flash and RAM regions can be defined. GDB uses this information for breakpoints, stack unwinding, and the memory display.

DeviceAPI.**memmapFromFile**(*filename*)

Loads the memory map XML from a file instead of an inline string. Returns `ERROR_OK` on success or `-1` on failure (file not found, read error, or XML parse error).

4.9.2 Memory map XML format

EBLink uses the standard GDB memory-map XML format, extended with EBLink-specific attributes and properties for flash sector management.

The GDB memory-map format is defined by the DTD at <http://sourceware.org/gdb/gdb-memory-map.dtd>. Full documentation of the standard format can be found in the GDB manual under *Memory Map Format* (<https://sourceware.org/gdb/current/onlinedocs/gdb/Memory-Map-Format.html>).

Flash memory attribute

eval (optional)

The byte value that represents an erased flash cell. Default is `0xFF`. Set this for devices where the erased state differs:

```
<memory type="flash" start="0x08000000" length="0x80000" eval="0x00">
```

Flash property elements

blocksize (mandatory)

Size of one flash sector in bytes. Accepts decimal or hexadecimal:

```
<property name="blocksize">0x800</property>
```

secstart (optional)

Sets the sector number for the first sector of this region. EBLink automatically counts up from this number for each subsequent sector. The counter carries over to the next flash region if `secstart` is not set there. Default is 0:

```
<property name="secstart">8</property>
```

sectype (optional)

Shorthand to set the cache and read behaviour for this region. Two independent flags control what EBLink does:

- **cache_mcu** — read the MCU flash into the sector cache so EBLink can detect changes and avoid unnecessary writes.
- **read_mcu** — serve GDB memory reads directly from the MCU instead of from the cache.

Value	cache_mcu	read_mcu	Flash operations	GDB reads
<i>(default)</i>	on	off	MCU content is read into cache; only changed sectors are written.	Served from cache.
virtual	off	off	Cache is treated as empty without reading the MCU; sector is always considered erased.	Served from cache (erased value).
nocache	off	on	Cache is treated as empty without reading the MCU; sector is always written.	Read directly from MCU.
read_through	on	on	MCU content is read into cache; only changed sectors are written.	Read directly from MCU.
ebskip	—	—	Sector is completely excluded from all flash operations.	Not accessible through EBLink.

```
<property name="sectype">nocache</property>
```

cache_mcu (optional)

Sets or clears the `cache_mcu` flag individually. Use `disabled` to clear it, any other value to set it. This is the same flag that `sectype` controls as part of its shorthand combinations:

```
<property name="cache_mcu">disabled</property>
```

read_mcu (optional)

Sets or clears the `read_mcu` flag individually. Use `disabled` to clear it, any other value to set it. This is the same flag that `sectype` controls as part of its shorthand combinations:

```
<property name="read_mcu">disabled</property>
```

4.9.3 Flash management

DeviceAPI.isChipErase()

Returns true if a mass erase is active. Used in `ebFlashErase()` when no `ebFlashEraseChip` is defined — EBLink then falls back to sector-by-sector erase and this function allows the script to detect whether that erase was triggered by a chip erase request or a normal sector erase.

DeviceAPI.sectorCount()

Returns the total number of flash sectors defined in the memory map.

DeviceAPI.setFlashEraseValue(*value*)

Sets the byte value that EBLink considers an “erased” cell. Default is 0xFF (standard NOR flash). Change this for devices with a different erase value (e.g. 0x00).

DeviceAPI.setFlashDontTrim(*value*)

When *value* is `true`, disables the automatic trimming of the flash write buffer to the erased value. By default EBLink strips trailing erased bytes from the end of each sector before calling `ebFlashWrite()` to avoid unnecessary writes.

DeviceAPI.modifyFlash(*addr*, *buffer*)

Writes *buffer* directly into the EBLink flash sector cache at address *addr*, without touching the target hardware. EBLink will include these modified sectors in the next flash programming cycle. Returns `ERROR_OK` or a negative error code.

Use this to patch data in flash before the normal write cycle:

```
local patch = EBbuffer(4);
patch.set(0, 0xDEADBEEF);
DeviceAPI().modifyFlash(0x080FFFC, patch);
```

4.10 TargetAPI

TargetAPI is a class registered by EBLink that controls the CPU state: halting, resuming, reading and writing registers, executing code, and resetting the device. Scripts instantiate it once at global scope:

```
targetApi <- TargetAPI()
```

All methods throw an integer `ERROR_xxx` exception on failure.

4.10.1 Target information

TargetAPI.getName()

Returns the device name string as reported by the target driver (e.g. "STM32F4") or `null` if no target is attached.

TargetAPI.getCpuName()

Returns the ARM CPU core name (e.g. "M0", "M3", "M4").

TargetAPI.getState()

Returns the current CPU state as one of the `TARGET_xxx` constants: `TARGET_UNKNOWN`, `TARGET_RUNNING`, `TARGET_HALTED`, `TARGET_RESET`.

4.10.2 Execution control

TargetAPI.halt()

Requests the CPU to halt. Throws on error.

TargetAPI.resume()

Resumes a halted CPU. Throws on error.

TargetAPI.poll()

Polls the target hardware and updates the internal state. Call this after `resume()` before reading `getState()` if you need an immediate state update.

TargetAPI.execute(addr, wait)

Writes `addr` to the program counter (PC register) and resumes the CPU. If `wait` is `true`, blocks until the CPU halts again. Useful for running a small RAM-resident flash algorithm:

```
// Upload algorithm to RAM, then execute it
intrfApi.writeBuf(algo_buf, 0x20000000);
targetApi.execute(0x20000000, true);
```

TargetAPI.reset(type)

Resets the device using one of the `xxx_RESET` constants:

```
targetApi.reset(SYSTEM_RESET);
```

For `USER_RESET`, the `ebResetUser()` script function is called instead of a hardware reset. After the reset the `ebResetPost()` function is called if it exists.

4.10.3 Register access

Register names are core-specific. For ARM Cortex-M the standard names are "r0" through "r15", "pc" (same as "r15"), "sp" (same as "r13"), "lr" (same as "r14"), and "xpsr".

`TargetAPI.readReg(name)`

Returns the 32-bit value of the named register. Throws on error or if the register name is unknown:

```
local pc = targetApi.readReg("pc");
printf("PC = 0x%08X\n", pc);
```

`TargetAPI.writeReg(name, value)`

Writes value to the named register. Throws on error or if the register name is unknown:

```
targetApi.writeReg("r0", 0x00000000); // clear R0
```

4.10.4 Example: read and print all core registers

```
function dump_regs()
{
    local regs = ["r0","r1","r2","r3","r4","r5","r6","r7",
                  "r8","r9","r10","r11","r12","sp","lr","pc","xpsr"];
    foreach(name in regs)
        printf("%-6s = 0x%08X\n", name, targetApi.readReg(name));
}
```

4.11 ServicesAPI

ServicesAPI is a class registered by EBLink, used to register EBLink services from within the ebServicesSetup() script function.

4.11.1 Service start behaviour

When ebServicesSetup() is **not defined** in the script, EBLink starts services as specified on the command line (e.g. -G for a GDB server).

As soon as the script calls ServicesAPI().addService() **at least once**, EBLink no longer starts any services on its own. From that point the script is fully responsible for starting all required services.

The script can retrieve the original CLI option string with cliSwitch() and use it as the base for the service options. This is useful when the script needs to add extra parameters (such as a DAP access port) while still honouring the options the user passed on the command line.

Note: Place any script-added options **after** the cliSwitch() string so they override the CLI defaults, not the other way around.

4.11.2 Example: dual-core GDB setup

The following pattern is taken from the STM32H7 dual-core script. When two cores are present, two separate GDB servers are started — one per core — each using the original -G CLI options with an overridden DAP access port:

```
function ebServicesSetup()
{
    // Single-core: let EBLink handle services via CLI as usual
    if(!hasDualCore)
        return ERROR_OK;

    try {
        local servicesApi = :: ServicesAPI()

        //
        //          type          name          options
        servicesApi.addService("GDB-server", "GDB_CM7", format("%s,ap=%d", cliSwitch(
→ "-G"), CM7_AP))
        servicesApi.addService("GDB-server", "GDB_CM4", format("%s,ap=%d", cliSwitch(
→ "-G"), CM4_AP))
    }
    catch(e) {
        if(catchString(e)) return ERROR_NOTIFIED;
        return e;
    }

    return ERROR_OK;
}
```

The cliSwitch("-G") call returns the option string of the **first** -G on the command line (e.g. "port=4242"). When a script manages its own dual-core setup via ebServicesSetup, it typically ignores any -G entries and builds its own service options. The ap= parameter is appended after the cliSwitch string so it takes precedence over any ap= value in the original CLI string.

4.11.3 Reference

`ServicesAPI.addService(type, name, options)`

Registers a service of the given `type` with the given `name` and `options` string. Throws an integer exception on failure.

- `type` — service type string (e.g. "GDB-server")
- `name` — instance name for this service
- `options` — option string passed to the service (format depends on type)

Calling this function at least once causes EBLink to hand full service control to the script — no CLI-specified services will be started automatically.

4.12 Script Debugger

EBlink includes a built-in Squirrel script debugger. It is activated through the verbosity flag and lets you trace, step through, and inspect script execution from the console without an external IDE.

4.12.1 Enabling the debugger

Flag	Effect
-v7	Enables the debug hook. EBlink prints each function call and return. <code>brkPt()</code> calls become active. Interactive stepping on call/return/exception events.
-v8	Additionally enables line-level debug info in the compiled bytecode. Stepping to the next source line becomes available. Produces more detailed output.

With `-v7` the output shows the function name and event type for each call/return. With `-v8` each source line is also reported.

4.12.2 Trace output format

While the debugger is active, EBlink prints a trace line for each event:

```
Squirrel debug      ebMain(c)[ 0]      {stm32.script}
Squirrel debug      ebFlashWrite(c)[ 0]    {stm32.script}
Squirrel debug      ebFlashWrite(r)[145]   {stm32.script}
```

Columns: debug/trace mode, function name (padded), event type in parentheses, line number in brackets, source file in braces.

Event types:

c	Function call (entering a function)
r	Function return (leaving a function)
e	Exception (a <code>throw</code> is about to fire)
l	Source line (only with <code>-v8</code>)

4.12.3 Script breakpoints

`brkPt()`

Inserts a breakpoint at the call site. Only active with `-v7` or higher; has no effect otherwise. When hit, EBlink prints the call stack with local variables and pauses for console input:

```
function ebFlashErase(sector, address)
{
    brkPt();           // execution stops here under -v7
    flash_unlock();
    // ...
}
```

4.12.4 Interactive commands

At a breakpoint or step-stop, EBLink shows the call stack and prompts:

```
== BREAKPOINT ==
*FUNCTION [ebFlashErase()] stm32.script line [42]
*FUNCTION [ebMain()] stm32.script line [15]
-LOCALS:
  [sector] INTEGER( 0 )
  [address] INTEGER( 134217728 )

Go, next Call, Return or Exception? [g,c,r,e]
```

Available commands:

Key	Level	Action
g	-v7	Go: continue until the next <code>brkPt()</code> or end
c	-v7	Call: run until the next function call
r	-v7	Return: run until the current function returns
e	-v7	Exception: run until the next <code>throw</code>
l	-v8 only	Line: run until the next source line

4.12.5 Call stack and locals

At every stop, EBLink prints the full call stack from the current function up to the top level, followed by all visible local variables with their types and values. Supported types: `INTEGER`, `FLOAT`, `STRING`, `NULL`, `TABLE`, `ARRAY`, `CLOSURE`, `INSTANCE`, `USERPOINTER`.

This information is the primary tool for understanding unexpected script behaviour — check the locals before stepping further to confirm the values are what you expect.

4.12.6 Tips

- Place `brkPt()` just before the operation you suspect is wrong, not at the top of a function, to reduce the number of steps needed.
- Use `-v7` for function-level tracing during initial development; switch to `-v8 only` when you need to trace individual lines.
- The debugger pauses the entire EBLink process — a GDB session connected to the same instance will time out if you leave a breakpoint open too long.

4.13 Addendum: Squirrel Extensions

EBlink modifies the Squirrel compiler and lexer in two places to make common MCU scripting patterns more concise. These extensions are not part of the upstream Squirrel language.

4.13.1 Multiline verbatim strings

Standard Squirrel verbatim strings (`@"..."`) suppress escape processing but do not allow embedded newlines. EBlink adds a double-prefix form `@@"..."` that allows newlines inside the string literal:

```
const mem_map = @@"<?xml version=\"1.0\"?>
<memory-map>
  <memory type=\"flash\" start=\"0x08000000\" length=\"0x80000\">
    <property name=\"pagesize\">0x800</property>
  </memory>
  <memory type=\"ram\" start=\"0x20000000\" length=\"0x20000\"/>
</memory-map>";
```

The string is copied verbatim including all embedded newline characters. Backslash escapes inside `@@"..."` are not interpreted (same as `@"..."` for single-line strings). This makes it practical to embed XML memory maps and other multi-line text directly in a script.

Note: `@"..."` — single-line verbatim: no escape processing, no newlines.

`@@"..."` — multiline verbatim: no escape processing, physical newlines in source are ignored (use `\n` for a newline in the value).

4.13.2 Compile-time constant expressions

Standard Squirrel `const` and `enum` values may only be integer, float, or string literals — no expressions are permitted. EBlink extends this so that an integer expression wrapped in parentheses is evaluated at compile time:

```
const BASE    = 0x40020000;
const CR1     = (BASE + 0x00);
const CR2     = (BASE + 0x04);
const ENABLE  = (1 << 0);
const MODE    = (3 << 2);

enum Flags {
  FLAG_A = (1 << 0),
  FLAG_B = (1 << 4)
}
```

Supported operators inside the parentheses:

Operator	Operation
<<	Left shift
>>	Right shift
*	Multiply
/	Divide
+	Add
-	Subtract / unary minus

Constraints:

- Only integer operands — no floats, no strings.
- An operand may be an integer literal or the name of a previously declared integer constant.
- Exactly one operator per expression. (A + B) is valid; (A + B + C) is a compile error. Chain constants to work around this:

```
const AB  = (A + B);  
const ABC = (AB + C);
```

- Bitwise operators (|, &, ^) are **not** supported inside ().

Pseudo BNF:

```
ConstExpr : '(' Operand ')'  
          | '(' Operand Operator Operand ')'  
Operand   : ['-'] ( IntegerLiteral | ConstantName )  
Operator  : '<<' | '>>' | '*' | '/' | '+' | '-'
```

4.14 API Quick Reference

DeviceAPI

- `isChipErase()`
- `memmap()`
- `memmapFromFile()`
- `modifyFlash()`
- `sectorCount()`
- `setFlashDontTrim()`
- `setFlashEraseValue()`

EBbuffer

- `EBbuffer()` (constructor)
- `access()`
- `allocate()`
- `format()`
- `get()`
- `getSize()`
- `getTrim()`
- `set()`
- `trim()`

InterfAPI

- `getDapAp()`
- `getName()`
- `getSpeed()`
- `getTapId()`
- `loadBin()`
- `loadString()`
- `read()`
- `readBuf()`
- `readDap()`
- `resetCtrl()`
- `setDapAp()`
- `setSpeed()`
- `targetVoltage()`
- `write()`
- `writeBuf()`

- `writeDap()`

ServicesAPI

- `addService()`

TargetAPI

- `execute()`
- `getCpuName()`
- `getName()`
- `getState()`
- `halt()`
- `poll()`
- `readReg()`
- `reset()`
- `resume()`
- `writeReg()`

Error handling

- *`catchString()`*
- *`catchThrowStr()`*
- *`errorToStr()`*

Global functions

- *`amregex()`*
- *`AnimateCursor()`*
- *`AnimateDone()`*
- *`astrsep()`*
- *`cliSwitch()`*
- *`FL_MAX()`*
- *`FL_MIN()`*
- *`Float2Integer()`*
- *`GetTickCount()`*
- *`IN_MAX()`*
- *`IN_MIN()`*
- *`include()`*
- *`Integer2Float()`*
- *`isInteger()`*
- *`isnull()`*

- *isScriptObject()*
- *isString()*
- *require()*
- *Sleep()*
- *strToNumber()*
- *UN_MAX()*
- *UN_MIN()*

Logging

- *debugf(fmt, ...)*
- *errorf(fmt, ...)*
- *noticef(fmt, ...)*
- *printf(fmt, ...)*
- *tracef(fmt, ...)*
- *userf(fmt, ...)*
- *warnf(fmt, ...)*
- *yesNof(default, fmt, ...)*

Debugger

- *brkPt()*

Symbols

?: Operator
 Operators, 32
 3 ways compare operator
 Operators, 33

A

amregex() (*built-in function*), 66
 AnimateCursor() (*built-in function*), 66
 AnimateDone() (*built-in function*), 66
 Arithmetic Operators
 Operators, 32
 array
 .append(), 54
 .apply(), 55
 .clear(), 55
 .extend(), 54
 .filter(), 55
 .find(), 55
 .insert(), 54
 .len(), 54
 .map(), 55
 .pop(), 54
 .push(), 54
 .reduce(), 55
 .remove(), 54
 .resize(), 54
 .reverse(), 55
 .slice(), 55
 .sort(), 54
 .top(), 54
 .toString(), 55
 Array constructor, 36
 array() (*built-in function*), 52
 Arrays, 38
 assert() (*built-in function*), 52
 assignment(=), 32
 astrsep() (*built-in function*), 66
 attributes
 Class, 45

B

Binding an environment to a function, 41
 Bitwise Operators
 Operators, 34
 block

statement, 27
 bool
 .toFloat(), 53
 .toInteger(), 53
 .toString(), 53
 break
 statement, 29
 brkPt() (*built-in function*), 86
 Built-in Functions, 52
 Global Symbols, 52

C

catchString() (*built-in function*), 61
 catchThrowStr() (*built-in function*), 61
 Class
 attributes, 45
 declaration, 43
 inheritance, 47
 instances, 46
 metamehtods, 49
 static variables, 44
 class
 .instance(), 56
 Class Attributes, 45
 Class Declaration, 43
 Class declaration
 statement, 30
 Class Instances, 46
 Class metamehtods, 49
 Classes, 43
 cliSwitch() (*built-in function*), 66
 clone, 36
 Comma operator
 Operators, 34
 comments, 21
 const
 statement, 31
 Constants, 50
 Constants & Enumerations, 50
 continue
 statement, 29
 control flow statements, 27

D

declaration
 Class, 43

delimiters, 20

DeviceAPI

- .isChipErase(), 80
- .memmap(), 79
- .memmapFromFile(), 79
- .modifyFlash(), 81
- .sectorCount(), 80
- .setFlashDontTrim(), 81
- .setFlashEraseValue(), 80

do/while

- statement, 28

E

EBbuffer

- (constructor), 75
- .access(), 75
- .allocate(), 75
- .format(), 75
- .get(), 75
- .getSize(), 75
- .getTrim(), 76
- .set(), 75
- .trim(), 76

ebFlashDone() *(built-in function)*, 72

ebFlashErase() *(built-in function)*, 71

ebFlashEraseChip() *(built-in function)*, 72

ebFlashStart() *(built-in function)*, 71

ebFlashWrite() *(built-in function)*, 71

ebGDBqRcmd() *(built-in function)*, 73

ebMain() *(built-in function)*, 71

ebResetPost() *(built-in function)*, 72

ebResetUser() *(built-in function)*, 72

ebServicesSetup() *(built-in function)*, 72

else

- statement, 27

enum

- statement, 31

Enumerations, 50

error() *(built-in function)*, 52

errorToStr() *(built-in function)*, 61

execution context, 25

Expression statement

- statement, 31

Expressions, 32

F

false, 27

FL_MAX() *(built-in function)*, 67

FL_MIN() *(built-in function)*, 67

float

- .tochar(), 52
- .tofloat(), 52
- .tointeger(), 52
- .tostring(), 52

Float2Integer() *(built-in function)*, 65

for

- statement, 29

foreach

- statement, 29

Free Variables, 42

function

- .call(), 56

- .pcall(), 56

Function calls, 41

Function Declaration, 39

Function declaration

- statement, 30

Function Default Paramaters, 40

Function with variable number of
paramaters, 40

Functions, 39

G

GDB

- monitor commands, 15

GetTickCount() *(built-in function)*, 66

Global Symbols

- Built-in Functions, 52

I

identifiers, 20

if

- statement, 27

if/else

- statement, 27

in operator

- Operators, 33

IN_MAX() *(built-in function)*, 67

IN_MIN() *(built-in function)*, 67

include() *(built-in function)*, 65

Inheritance, 47

inheritance

- Class, 47

instance

- .getclass(), 56

instanceof operator

- Operators, 33

instances

- Class, 46

integer

- .tochar(), 52
- .tofloat(), 52
- .tointeger(), 52
- .tostring(), 52

Integer2Float() *(built-in function)*, 65

InterfAPI

- .getDapAp(), 77
- .getName(), 77
- .getSpeed(), 77
- .getTapId(), 77
- .loadBin(), 78
- .loadString(), 78
- .read(), 78
- .readBuf(), 78
- .readDap(), 77
- .resetCtrl(), 77

- .setDapAp(), 77
- .setSpeed(), 77
- .targetVoltage(), 77
- .write(), 78
- .writeBuf(), 78
- .writeDap(), 77

introduction, 1

isInteger() (*built-in function*), 65

isNull() (*built-in function*), 65

isScriptObject() (*built-in function*), 65

isString() (*built-in function*), 65

K

keywords, 20

L

Lambda Expressions, 41

lexical structure, 20

literals, 20

Local variables declaration

- statement, 30

Logical operators

- Operators, 33

Loops, 29

M

metamethods

- Class, 49

monitor commands, 15

- clrbp, 15
- DAP, 15
- execute, 15
- is_running, 15
- ReadAP, 15
- ReadDP, 15
- reg, 15
- reset, 15
- semihosting, 15
- servicePollInterval, 15
- sleep, 15
- WriteAP, 16
- WriteDP, 15
- WriteMemAP, 16
- writeu32, 15

N

new slot(<-), 32

numeric literals, 20

O

Operators, 32

- ?: Operator, 32
- 3 ways compare operator, 33
- Arithmetic Operators, 32
- Bitwise Operators, 34
- Comma operator, 34
- in operator, 33

- instanceof operator, 33
- Logical operators, 33
- Operators precedence, 34
- Relational Operators, 33
- typeof operator, 34

operators, 20

Operators precedence

- Operators, 34

other tokens, 20

P

print() (*built-in function*), 52

R

Relational Operators

- Operators, 33

require() (*built-in function*), 65

return

- statement, 30

S

ServicesAPI

- .addService(), 85

Sleep() (*built-in function*), 66

Slot Creation(table), 37

Slot Deletion(table), 37

statement

- block, 27
- break, 29
- Class declaration, 30
- const, 31
- continue, 29
- do/while, 28
- else, 27
- enum, 31
- Expression statement, 31
- for, 29
- foreach, 29
- Function declaration, 30
- if, 27
- if/else, 27
- Local variables declaration, 30
- return, 30
- switch, 28
- throw, 31
- try/catch, 30
- while, 28

statements, 27

Static variables, 44

static variables

- Class, 44

string

- .find(), 53
- .len(), 53
- .slice(), 53
- .toFloat(), 53
- .tointeger(), 53
- .tolower(), 53

- `.toString()`, 53
 - `.toupper()`, 53
- string literals, 20
- `strToNumber()` (*built-in function*), 65
- switch
 - statement, 28

T

- table
 - `.clear()`, 54
 - `.filter()`, 54
 - `.keys()`, 54
 - `.len()`, 54
 - `.toString()`, 54
 - `.values()`, 54
- Table Constructor, 35
- Table with JSON syntax, 35
- Tables, 37
- Tail Recursion, 42
- TargetAPI
 - `.execute()`, 82
 - `.getCpuName()`, 82
 - `.getName()`, 82
 - `.getState()`, 82
 - `.halt()`, 82
 - `.poll()`, 82
 - `.readReg()`, 83
 - `.reset()`, 82
 - `.resume()`, 82
 - `.writeReg()`, 83
- throw
 - statement, 31
- true, 27
- true and false, 27
- try/catch
 - statement, 30
- `type()` (*built-in function*), 52
- typeof operator
 - Operators, 34

U

- `UN_MAX()` (*built-in function*), 67
- `UN_MIN()` (*built-in function*), 67

W

- while
 - statement, 28